SportsOS: An AI-Powered Sports ERP for IIT (ISM) Dhanbad

Thesis Mid-Evaluation Report

submitted in partial fulfilment of the requirements for the degree of

Integrated Master of Technology

in

Mathematics and Computing

Submitted by

Anup Kumar

Admission No.: 22JE0143

Under the guidance of

Prof. Scindhiya Laxmi

Department of Mathematics and Computing

Indian Institute of Technology (Indian School of Mines), Dhanbad

Dhanbad, Jharkhand, India

13 November 2026

ABSTRACT

Sports administration at IIT (ISM) Dhanbad depends on paper registers, spreadsheets and informal messaging, which leaves facility usage, attendance and equipment stock without real-time visibility or a reliable audit trail. This thesis designs and develops SportsOS, a role-aware sports Enterprise Resource Planning system that unifies facility booking, verified attendance, equipment inventory, tournaments, complaints, announcements and analytics for the institute's sports ecosystem.

The work is positioned in Software Engineering and Information Systems. Requirements were derived from the approved proposal and modelled as 18 stakeholder roles, 70 permissions and four scope types. The system is built as a TypeScript monorepo with a Next.js 15 web client, an Expo SDK 54 mobile client, and a NestJS 11 REST API over a 31-entity PostgreSQL schema managed with Prisma. Its main engineering contributions are: scope-aware, time-bounded role-based access control with two-person approval for privileged roles; proof-of-presence attendance combining time-based one-time passwords, GPS geofencing and device binding; database-enforced consistency for bookings and stock; and a SHA-256 hash-chained audit ledger. An optional retrieval-augmented assistant operates on permission-filtered aggregate data.

For this mid-evaluation, the author re-executed the full verification suite on 12 November 2026: 71 of 71 unit tests, 83 of 83 API smoke checks, 200 of 200 role-based access-control and workflow checks across all 18 roles, 21 of 21 browser checks, and a set of targeted checks run by the author all passed. The evaluation also confirms the limitations: there is no password-reset flow and no shared cache; the mobile client has been type-checked but not device-tested; the AI layer has been verified only in offline mode; all data is synthetic; and load, penetration and user testing remain to be done. The evaluation is therefore preliminary, and the remaining months are planned for stakeholder validation and hardening.

Keywords: sports management information system, enterprise resource planning, role-based access control, verified attendance, geofencing, web application security, NestJS, Next.js, PostgreSQL.

TABLE OF CONTENTS

Abstract ..... **i**

List of Figures ..... **iv**

List of Tables ..... **v**

List of Abbreviations ..... **vi**

CHAPTER 1: INTRODUCTION **1**

1.1 Introduction 1

1.2 Institutional Background and Sports Ecosystem 1

1.3 Existing Operational Context and Problem Statement 2

1.4 Motivation 2

1.5 Research Objectives and Scope 2

1.6 Expected Contributions and Report Organisation 3

CHAPTER 2: LITERATURE REVIEW AND RELATED WORK **4**

2.1 Digital Transformation in Sports Administration 4

2.2 Sports Management Information Systems and Facility Management 4

2.3 Enterprise Systems and Multi-Stakeholder Design 4

2.4 Role-Based Access Control and Web Security 5

2.5 Comparative Analysis and Research Gap 5

CHAPTER 3: REQUIREMENTS ENGINEERING AND SYSTEM ANALYSIS **7**

3.1 Stakeholders, Hierarchy and Roles 7

3.2 Functional and Non-Functional Requirements 8

3.3 Use Cases, User Stories and Workflows 8

3.4 Constraints and Development Methodology 9

CHAPTER 4: SYSTEM ARCHITECTURE AND DATABASE DESIGN **11**

4.1 System Overview and Architectural Approach 11

4.2 Frontend, Backend and API Design 11

4.3 Database Architecture and Entity Relationship Diagram 12

4.4 Authentication and RBAC Design 13

4.5 Data Flow, Security and Deployment 15

CHAPTER 5: IMPLEMENTATION AND DEVELOPMENT **17**

5.1 Development Environment, Stack and Monorepo 17

5.2 Frontend and Backend Implementation 17

5.3 Authentication, User Management and RBAC 18

5.4 Clubs, Facilities and Booking 19

5.5 Verified Attendance 20

5.6 Equipment and Inventory 21

5.7 Tournaments, Complaints, Notifications and Analytics 22

5.8 Mobile Application and AI Features 22

5.9 Module-Wise Implementation Status 23

CHAPTER 6: SOFTWARE TESTING AND EVALUATION **24**

6.1 Testing Strategy 24

6.2 Unit and Integration Testing 24

6.3 Functional, Authentication and Authorisation Testing....25
6.4 Database, API, End-to-End and Security Testing....26
6.5 Preliminary Evaluation....26
CHAPTER 7: RESULTS, LIMITATIONS AND FUTURE WORK....27
7.1 Current Status and Achievement Against Objectives....27
7.2 Demonstration and Engineering Contributions....27
7.3 Current Limitations and Technical Challenges....28
7.4 Future Enhancements and Revised Roadmap....28
7.5 Conclusion....29
REFERENCES....31
APPENDIX A: REPOSITORY STRUCTURE....33
APPENDIX B: SELECTED API DOCUMENTATION....34
APPENDIX C: DATABASE ENTITY CATALOGUE....36
APPENDIX D: IMPLEMENTATION EVIDENCE MATRIX....37
APPENDIX E: ADDITIONAL SCREENSHOTS....38
APPENDIX F: DETAILED TEST CASES....42

LIST OF FIGURES

Figure 3.1: Use case diagram of principal SportsOS actors and use cases.....9
Figure 4.1: System architecture of SportsOS (versions verified from package manifests)….11
Figure 4.2: Entity relationship diagram of core SportsOS entities (from schema.prisma)....13
Figure 4.3: Authentication sequence (login and subsequent request).....14
Figure 4.4: RBAC enforcement pipeline from request to handler.....15
Figure 4.5: Level-1 data flow diagram.....15
Figure 4.6: Deployment architecture as configured in docker-compose.yml.....16
Figure 5.1: Sports Officer dashboard with KPIs and approval queue (synthetic data).....18
Figure 5.2: Facility booking workflow (from bookings.service.ts).....19
Figure 5.3: Attendance check-in validation workflow (attendance.service.ts).....20
Figure 5.4: Coach console with a live verified-attendance session.....21
Figure 5.5: Equipment issue and return workflow.....21
Figure 7.1: Revised development roadmap.....29
Figure E.1: Complaint lifecycle state machine (complaints.service.ts).....38
Figure E.2: Knock-out bracket with seeded byes and automatic advancement (synthetic data).....38
Figure E.3: Administrator view of roles and the editable role × permission matrix (synthetic data).....39
Figure E.4: Facility page with the daily availability grid (synthetic data).....39
Figure E.5: Analytics page: participation, utilisation and forecast views (synthetic data)….40
Figure E.6: SportsOS AI assistant page (offline retrieval mode) (synthetic data).....40
Figure E.7: Inventory view with stock levels and low-stock alerts (synthetic data).....41
Figure E.8: Student dashboard (synthetic data).....41

LIST OF TABLES

Table 1.1: Research objectives from the approved proposal.....3

Table 2.1: Comparison of reviewed work with SportsOS (claims limited to what each source reports).....5

Table 3.1: Implemented roles and default permission counts (measured).....7

Table 3.2: Functional requirements and current status.....8

Table 3.3: Non-functional requirements (ISO/IEC 25010 characteristics).....8

Table 4.1: Security controls mapped to ASVS 5.0.0 areas (indicative).....16

Table 5.1: Technology stack (versions verified from package manifests).....17

Table 5.2: Module-wise implementation status at mid-evaluation.....23

Table 6.1: Test levels and results of the 12 November 2026 run.....24

Table 6.2: Representative test cases (complete tables in Appendix F).....25

Table 7.1: Achievement against the proposal's objectives.....27

Table 7.2: Known limitations at mid-evaluation.....28

Table A.1: Source size by area (lines, measured with wc -l on .ts/.tsx/.mjs/.cjs/.prisma/.sql files).....33

Table B.1: API surface by module (counted from controller decorators).....34

Table B.2: Selected endpoints.....34

Table C.1: All 31 Prisma models grouped by domain.....36

Table C.2: Seeded synthetic data volumes (run of 12 Nov 2026).....36

Table D.1: Requirement → implementation → verification evidence.....37

Table F.1: Unit test suites (Jest), 71 tests, all passing.....42

Table F.2: Functional test cases: booking, inventory and complaints (results from the smoke and e2e runs).....42

Table F.3: Authentication test cases.....43

Table F.4: Authorisation and RBAC test cases.....43

Table F.5: Database test cases (author's checks).....44

Table F.6: Attendance verification test cases (author's run against a live session).....44

LIST OF ABBREVIATIONS

Abbreviation	Expansion
API	Application Programming Interface
ASVS	Application Security Verification Standard (OWASP)
CORS	Cross-Origin Resource Sharing
CSP	Content Security Policy
DFD	Data Flow Diagram
DTO	Data Transfer Object
e2e	End-to-end (testing)
ERD	Entity Relationship Diagram
ERP	Enterprise Resource Planning
GPS	Global Positioning System
HMAC	Hash-based Message Authentication Code
HOTP	HMAC-based One-Time Password
IST	Indian Standard Time
JWT	JSON Web Token
LLM	Large Language Model
MIS	Management Information System
NAT	Network Address Translation
ORM	Object-Relational Mapping
OTP	One-Time Password
OWASP	Open Worldwide Application Security Project
PESO	Sports Officer (abbreviation as used in the proposal)
QR	Quick Response (code)
RAG	Retrieval-Augmented Generation
RBAC	Role-Based Access Control
REST	Representational State Transfer
SPEC	Sports & Physical Education Center
SSE	Server-Sent Events
SW	Students' Welfare
TOTP	Time-based One-Time Password
UAT	User Acceptance Testing

CHAPTER 1

INTRODUCTION

1.1 Introduction

Institutional sport generates a steady stream of records: practice sessions, ground allocations, equipment loans, fixtures and complaints. Several offices must create, approve, reconcile and report on these records. When they live in paper registers, spreadsheets and messaging groups, the institution cannot see its operations in real time, audit them afterwards or plan from evidence. This thesis addresses that problem for IIT (ISM) Dhanbad through **SportsOS**, a role-aware sports Enterprise Resource Planning (ERP) system. SportsOS consists of a web application, a mobile application and a REST API over one relational database.

The thesis belongs to Software Engineering and Information Systems, not machine learning. AI features are treated as an optional layer on the transactional core and are reported only as far as they have been verified. This report documents the state of the project on 13 November 2026, three months into the nine-month plan of the approved proposal [1]. All test results in Chapter 6 were re-executed by the author on 12 November 2026 against a freshly migrated and seeded PostgreSQL 16 database.

1.2 Institutional Background and Sports Ecosystem

Sport at IIT (ISM) is administered under the Dean (Students' Welfare) through the Sports & Physical Education Center (SPEC). SPEC's pages describe its remit as including intramural competitions such as the **Annual Sports Meet** and the **Inter-Hostel General Championship** [2].

SPEC's facilities page lists the following infrastructure [3]:

- a Lower Ground with a football ground and a floodlit eight-lane, 400 m track;
- a Sports Complex with floodlit basketball, volleyball and tennis courts;
- a Students Activity Centre with a gymnasium, four synthetic badminton courts, a yoga centre, a boxing ring and a billiards room;
- a 50 m swimming pool.

The two institute pages named in the brief [4], [5] could not be reached from the isolated evaluation environment, and the SPEC pages are undated. Institutional statements are therefore limited to [2], [3] and should be re-confirmed with SPEC. No sports personnel are named in this report.

1.3 Existing Operational Context and Problem Statement

The proposal identifies five operational areas that currently run on registers, spreadsheets and messaging groups [1]:

- facility allocation;
- participation and attendance;
- tournaments;
- equipment issue and return;
- administrative coordination.

This baseline is the author's stated premise and has not yet been measured.

Problem statement. Fragmented, offline sports operations leave administrators without real-time visibility of facility usage, attendance and stock. The records that do exist are easy to manipulate (for example, proxy attendance) and hard to audit. The institute needs an integrated, role-aware web and mobile ERP that turns these operations into auditable data, while giving each stakeholder access only to what their role requires.

1.4 Motivation

Recent research on sports digitalisation has examined single solutions for single stakeholders, and it calls for multi-stakeholder work [6]. Integrated sports-facility systems are associated with better service quality, yet they remain little studied [7]. In multi-role systems, broken access control is the leading web-application risk [8], [9]. A campus sports ERP with eighteen roles is therefore both a practical need and a realistic test bed for rigorous role-based access control.

1.5 Research Objectives and Scope

The six objectives of the approved proposal are retained unchanged (Table 1.1).

Table 1.1: *Research objectives from the approved proposal*

ID	Objective	Measured by
O1	Centralise facility booking, tournaments, attendance, inventory and events in one platform	Single schema and API (Ch. 4–5)
O2	Automate scheduling, approvals, record management and reporting	Workflows, scheduled jobs, notifications
O3	Role-based access control for all stakeholders	RBAC design; 18-role tests (Ch. 4, 6)
O4	Real-time analytics and reporting	Analytics endpoints and dashboards
O5	One portal for the student experience	Student web and mobile functions
O6	Scalable, secure, future-ready system able to host AI features	Architecture, security, AI layer and limits

In scope: authentication; scoped, time-bound RBAC; facilities and booking; clubs; OTP- and GPS-verified attendance; QR-based inventory; tournaments; complaints; notifications and announcements; analytics; a mobile client; an optional AI assistant; and institutional data import.

Out of scope: payments; institute SSO or central-ERP integration; and real student data before institutional approval. All current data is synthetic.

1.6 Expected Contributions and Report Organisation

The thesis is expected to contribute:

- a requirements model of the sports ecosystem with 18 roles, 70 permissions and four scope types;
- a 31-entity schema and architecture with concurrency-safe booking and inventory and a hash-chained audit ledger;
- a working prototype across web, mobile and API;
- a verification method that tests every protected endpoint class against every role.

Chapter 2 reviews related work. Chapter 3 covers requirements. Chapter 4 presents the architecture and database, Chapter 5 the implementation, Chapter 6 testing, and Chapter 7 results and future work.

CHAPTER 2

LITERATURE REVIEW AND RELATED WORK

2.1 Digital Transformation in Sports Administration

Vial describes digital transformation as technology-triggered change that also reshapes processes and structures [10]. For a sports office, this means that digitising a register is not enough: the approvals, escalations and reporting built around it must be redesigned too.

Haffner, Oshri and Kotlarsky review information-systems research on sports digitalisation through a stakeholder lens [6]. They find that most studies consider one solution and one stakeholder group, and they call for multi-stakeholder perspectives. SportsOS adopts this perspective: a single booking involves a student, an approving officer, gate security and leadership reading utilisation data.

2.2 Sports Management Information Systems and Facility Management

Saputra and Asmawati systematically review integrated management information systems for sports facilities (seven articles, 2018–2023) [7]. They identify inventory, reservations, cleanliness supervision and performance reporting as the integrated components, and they link integration to better user experience and service quality.

Shahrom and Mohamad Sukri build a swimming-facility management system for a Malaysian university sport centre, with bookings, payments, RBAC and two-factor authentication [11]. Their OTP secures login, whereas SportsOS uses an OTP to prove physical presence.

A problem common to integrated facility systems is **consistency under concurrency**: two users must not obtain the last slot or the last ball. SportsOS handles this with database advisory locks and conditional updates [12].

2.3 Enterprise Systems and Multi-Stakeholder Design

Enterprise systems integrate an organisation's information flows into one database and one process model [13], [14]. SportsOS inherits two of their properties:

- **a single shared data model**, so that bookings, sessions, tournaments and complaints all reference one `Facility`;

- **process orientation**, so that every approval or escalation is a recorded state transition.

Stakeholder theory [15] adds two tensions that the SportsOS role model encodes explicitly. First, seniority does not imply operational authority: a Director sees dashboards but does not approve bookings. Second, student office-bearers act only within a limited scope, such as one hostel or one sport.

2.4 Role-Based Access Control and Web Security

The NIST RBAC model defines flat, hierarchical, constrained and symmetric RBAC [16], and it was later formalised as a proposed standard [17]. SportsOS implements core RBAC with validated session role selection. It deliberately omits role inheritance, and it adds two refinements:

- **scoped assignments**, which apply to a club, hostel, sport or facility;
- **time-bounded assignments**, which lapse at a set date.

Separation of duty appears in specific workflows: authors cannot approve their own proposals, and privileged roles need approval by a second person.

For security, the design draws on OWASP ASVS 5.0.0 [9] and the OWASP Top 10 [8]. The resulting controls are bcrypt password hashing [18], JWT bearer tokens [19], strict input whitelisting, rate limiting, security headers, and TOTP [20], [21] used as a proof-of-presence factor.

2.5 Comparative Analysis and Research Gap

Table 2.1: *Comparison of reviewed work with SportsOS (claims limited to what each source reports)*

Work	Setting	Booking	Attendance verification	Inventory	Access control	Audit trail
Haffner et al. 2025 [6]	IS review, sports digitalisation	—	—	—	Stakeholder lens	—
Saputra & Asmawati 2025 [7]	Review, sports facilities	Reported	Not reported	Reported	Not reported	Not reported
Shahrom & Mohamad Sukri 2025 [11]	University pool	Yes (+ payments)	No (OTP for login)	Not reported	RBAC + 2FA	Not reported
SportsOS (this work)	Institute-wide campus sports	Concurrency -safe	TOTP + GPS + device binding	Ledger + QR	Scoped, time-bound RBAC (18 × 70)	SHA-256 hash chain

Research gap. The literature supplies principles but no integrated, verifiable system for an academic sports ecosystem that meets four conditions:

1. student office-bearers hold only scoped authority;
2. attendance must be provably physical;
3. booking and stock records stay consistent and auditable;
4. authorisation is tested systematically across every role.

SportsOS addresses this gap.

CHAPTER 3

REQUIREMENTS ENGINEERING AND SYSTEM ANALYSIS

3.1 Stakeholders, Hierarchy and Roles

Requirements were derived from the proposal [1] and written to the ISO/IEC/IEEE 29148 criteria (necessary, unambiguous, verifiable) [22]. Quality attributes follow ISO/IEC 25010 [23]. Each stakeholder in the proposal's governance structure became one role, giving 17 organisational roles plus a System Administrator.

The proposal places the Director, the Dean (Students' Welfare) and the President of SPEC at the top. The Sports Officer (PESO), the General Secretary and the Event Committee sit below them, followed by staff, council roles and students. This hierarchy is the **author's requirements model**, not a confirmed official chart. The system therefore uses it only to draw the organisation page and never to grant permissions. Table 3.1 lists the roles, with default permission counts measured from `/api/auth/me` on 12 November 2026.

Table 3.1: *Implemented roles and default permission counts (measured)*

Role	Responsibility in SportsOS	Scope	Perms
DIRECTOR	Read-only institute dashboards	Global	18
DEAN_SW	Analytics, escalated complaints, privileged-role approval	Global	27
SPEC_PRESIDENT	Approves event proposals and announcements	Global	22
SPORTS_OFFICER	Facilities, bookings, inventory, tasks, analytics	Global	61
GENERAL_SECRETARY	Student lead; submits proposals	Global	16
EVENT_COMMITTEE	Tournaments, fixtures, results	Global	18
COACH	Attendance and rosters of coached clubs only	Coached clubs	17
GROUND_STAFF	Facility status, schedule, tasks	Optional facility	16
EQUIPMENT_MANAGER	Issue/return, stock ledger, requests	Global	17
SECURITY_STAFF	Gate-pass verification, minimal entry log	Global	11
OPERATIONS_STAFF	Operational tasks	Global	15
SPORTS_COUNCIL	Council functions, participation data	Global	16
SPORT_COORDINATOR	Sessions and rosters for one sport	Sport	17
HOSTEL_SECRETARY	Registers own hostel's teams	Hostel	14
MEDIA_NOMINEE	Drafts announcements	Global	20
CLUB_MEMBER	Club bookings, verified check-in	Club	13
STUDENT	Casual bookings, complaints, own records	Self	12
ADMIN	Users, roles, matrix, imports (no operational approvals)	Global	22

3.2 Functional and Non-Functional Requirements

Table 3.2: *Functional requirements and current status*

ID	Requirement	Obj.	Status
FR-01	Email/password sign-in; institute-domain self-registration yields STUDENT only	O3	Implemented, tested
FR-02	Self-service password reset	O5	Not implemented
FR-03	Scoped, time-bound role assignment; privileged roles need a second approver	O3	Implemented, tested
FR-04	Run-time editable role→permission matrix with lock-out protection	O3	Implemented, tested
FR-05	Facility catalogue, availability grid, status changes by staff	O1	Implemented, tested
FR-06	Validated bookings, auto-approved or queued; QR gate pass verified by security	O1, O2	Implemented, tested
FR-07	Coach-run sessions; check-in with rotating OTP and GPS inside the geofence; reasoned corrections	O1	Implemented, tested
FR-08	Equipment issue/return by QR with damage and loss; ledger; low-stock and overdue alerts	O1, O2	Implemented, tested
FR-09	Tournament proposals, hostel-scoped team registration, fixtures, results	O1	Implemented, tested
FR-10	Complaint lifecycle with escalation and attachments	O2	Implemented, tested
FR-11	Notifications, club announcements, approval-gated official announcements	O5	Implemented, tested
FR-12	KPIs, participation, utilisation, forecast, equipment wear, attendance integrity	O4	Implemented, tested (synthetic data)
FR-13	Assistant answering questions and drafting reports from permission-filtered data	O6	Partial: offline mode tested
FR-14	Mobile app for check-in, QR scanning, bookings and gate passes	O5	Partial: type-checked only
FR-15	Tamper-evident audit ledger with verification	O2, O6	Implemented, tested

Table 3.3: *Non-functional requirements (ISO/IEC 25010 characteristics)*

ID	Characteristic	Requirement	Status
NFR-01	Confidentiality	Hashes, OTP secrets, device IDs never returned	Verified
NFR-02	Authenticity	Valid JWT on every protected route; role changes apply on next request	Verified
NFR-03	Integrity	No double-booking; stock never negative under concurrency	Verified at small scale
NFR-04	Accountability	State changes hash-chained in the same transaction	Verified
NFR-05	Maintainability	Typed end to end; shared role definitions; lint-clean	Verified
NFR-06	Portability	Containerised API, web and database	Configured, not executed
NFR-07	Performance	Pages respond within 2 s for 500 concurrent users	Not tested
NFR-08	Privacy	Minimal data to security staff; AI never sees personal records	Verified

3.3 Use Cases, User Stories and Workflows

Figure 3.1 shows the principal actors and use cases, all of which exist in the implementation. Requirements were also written as user stories [24], each with an acceptance criterion that is tested in Chapter 6. Two examples follow.

- *As a coach, I want attendance that cannot be marked by proxy.* A check-in is accepted only with the current TOTP, GPS accuracy of at most 100 m, and a position inside the geofence.
- *As a hostel secretary, I want to register only my hostel's team.* Registration for another hostel, or of a non-resident player, is refused.

The booking, attendance and equipment workflows are modelled as activity diagrams in Chapter 5.

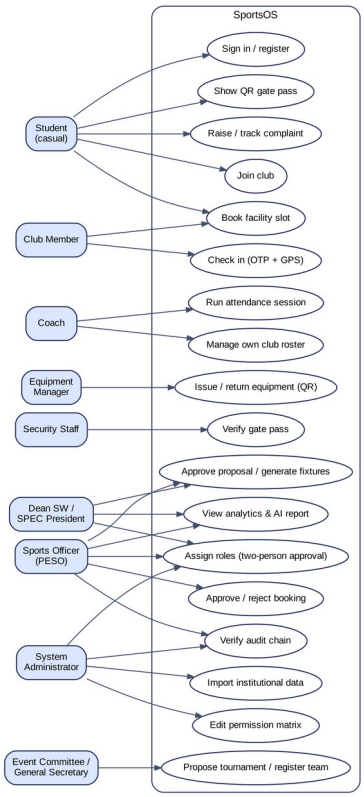


Figure 3.1: *Use case diagram of principal SportsOS actors and use cases*

3.4 Constraints and Development Methodology

The project operates under five constraints:

- no real personnel or student data may be used without authorisation;
- campus users share NAT addresses, so rate limits are applied per account;
- GPS accuracy is variable, so the geofence tolerates up to 25 m of reported inaccuracy and rejects fixes worse than 100 m;
- no institute server has been allocated yet;
- no AI API keys are available, so the system must work without them.

The process is iterative and incremental [25] and is framed as **design science research** [26]: an artefact is built for an identified organisational problem and then evaluated rigorously. Each increment followed the same cycle: refine requirements; extend the schema and migration; implement the service and controller with permission decorators; build the web and mobile UI; add tests; and re-run the full suite.

CHAPTER 4

SYSTEM ARCHITECTURE AND DATABASE DESIGN

4.1 System Overview and Architectural Approach

SportsOS is a three-tier system (Figure 4.1). A Next.js web client and an Expo mobile client call a NestJS REST API, which persists all state in PostgreSQL through Prisma. A shared TypeScript package defines roles, permissions and wire types once for both server and client. The Claude API and Pinecone are optional, and neither was configured in the evaluated setup.

The API is a **modular monolith** [27] of 20 application modules, each with a controller, a service and DTOs. A monolith was chosen for three reasons:

- the modules share transactions; for example, a booking and its audit entry must commit together;
- the load of a single institute does not justify microservices;
- one deployable unit suits a small campus IT team.

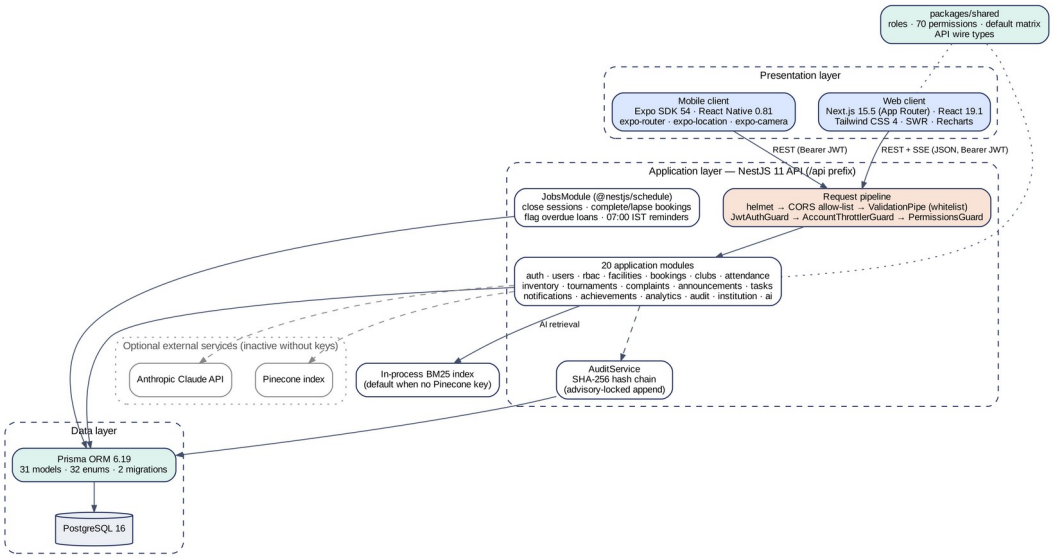


Figure 4.1: *System architecture of SportsOS (versions verified from package manifests)*

4.2 Frontend, Backend and API Design

Frontend. The web client uses the Next.js 15 App Router [28]. Authenticated pages sit in an `(app)` route group whose shell provides permission-driven navigation, a notification bell and an active-role switcher. The production build has 28 routes. SWR hooks call a

small `fetch` wrapper that attaches the bearer token and the `X-Active-Role` header. Hiding a menu item is only a usability aid: every action is enforced again on the server.

Backend. `main.ts` installs `helmet`, a CORS allow-list and a global `ValidationPipe` with whitelisting, so unknown request fields are rejected with HTTP 400. Three global guards then run in order: authenticate (`JwtAuthGuard`), rate-limit per account, and authorise (`PermissionsGuard`). Services hold the business rules and use Prisma interactive transactions for any change that spans several rows [29].

API. The API is resource-oriented REST with **143 route handlers**. State transitions are modelled as sub-resource POSTs, such as `POST /bookings/:id/approve` and `POST /inventory/issues/:id/return`, and each one maps to a single audit entry. One endpoint streams Server-Sent Events: the AI answer stream. Appendix B lists the full API surface.

4.3 Database Architecture and Entity Relationship Diagram

The Prisma schema defines **31 models and 32 enums** across two migrations, covering identity and RBAC, facilities and bookings, clubs, verified attendance, inventory, tournaments, engagement and governance. Figure 4.2 shows the core entities; the full catalogue is in Appendix C. The database itself enforces integrity in four ways:

- **Unique constraints.** For example, `@@unique([sessionId, userId])` allows one attendance record per student per session.
- **Advisory locks** [12]. Booking takes a lock keyed by facility before checking for overlaps, and audit appends take a constant lock.
- **Conditional updates.** Stock changes use `UPDATE … WHERE availableQty >= n` inside `applyStock`, which also writes a ledger row with the resulting balances.
- **History instead of deletion.** Role assignments, inventory transactions and attendance corrections are append-only.

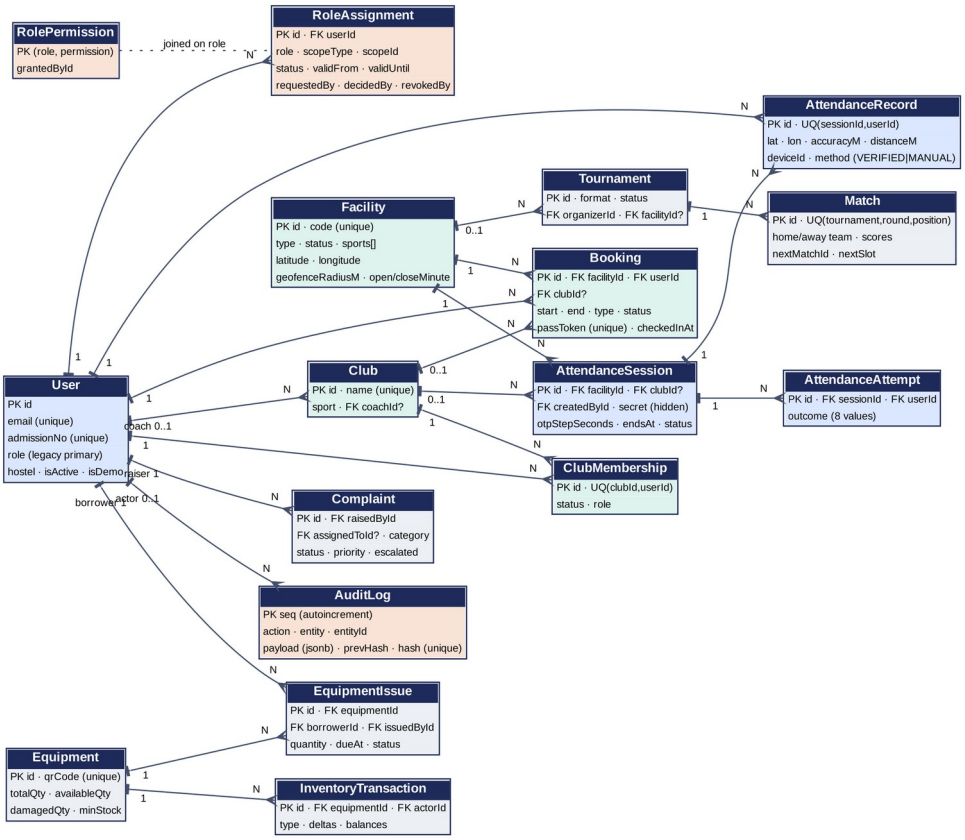


Figure 4.2: *Entity relationship diagram of core SportsOS entities (from schema.prisma)*

4.4 Authentication and RBAC Design

Authentication (Figure 4.3). Login is throttled per IP and email. It always performs a bcrypt comparison, using a dummy hash for unknown emails, so response timing does not reveal which accounts exist. A successful login issues a 12-hour JWT. On every request the guard verifies the token and **reloads the principal from the database**, ignoring the role claim inside the token, so revocations and deactivations take effect immediately. The web client stores the token in `localStorage`; the mobile client uses `expo-secure-store`.

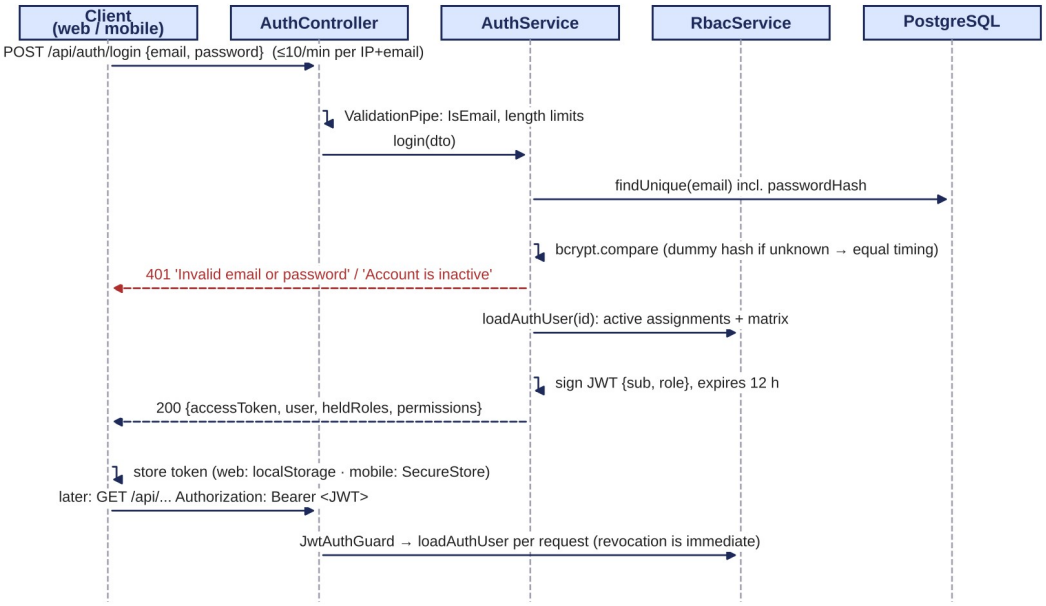


Figure 4.3: *Authentication sequence (login and subsequent request)*

RBAC (Figure 4.4). An assignment links a user to one of 18 roles with three attributes:

- a scope: `GLOBAL`, `CLUB`, `HOSTEL`, `SPORT` or `FACILITY`;
- a validity window;
- a status: pending, active, rejected, revoked or expired.

The role→permission matrix (70 permissions and 346 default grants) is stored in the database, cached for 30 seconds and editable at run time. A pure, unit-tested function resolves the permissions for each request in three steps:

1. Keep only the effective assignments.
2. Check that any `X-Active-Role` header names a role the user actually holds; otherwise refuse with 403.
3. Take the union of the remaining roles' matrix rows.

Route guards check permissions, and services also check **scope** through `assertCanIn`. A coach's club permissions apply only to clubs they coach. Roles that require a scope never grant permissions globally. Privileged roles stay pending until a second person approves them, and administrators cannot be locked out of matrix management.

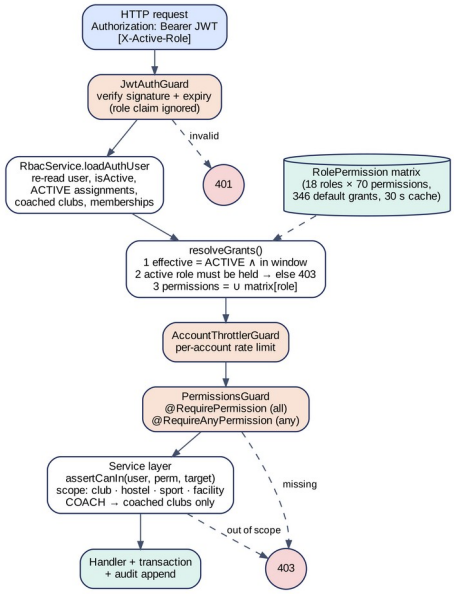


Figure 4.4: *RBAC enforcement pipeline from request to handler*

4.5 Data Flow, Security and Deployment

Figure 4.5 is a level-1 DFD. The booking, attendance and inventory processes append to the audit ledger (D5), and analytics only reads operational data. Table 4.1 maps the security controls to ASVS 5.0.0 areas. The mapping is indicative and does not constitute an ASVS assessment.

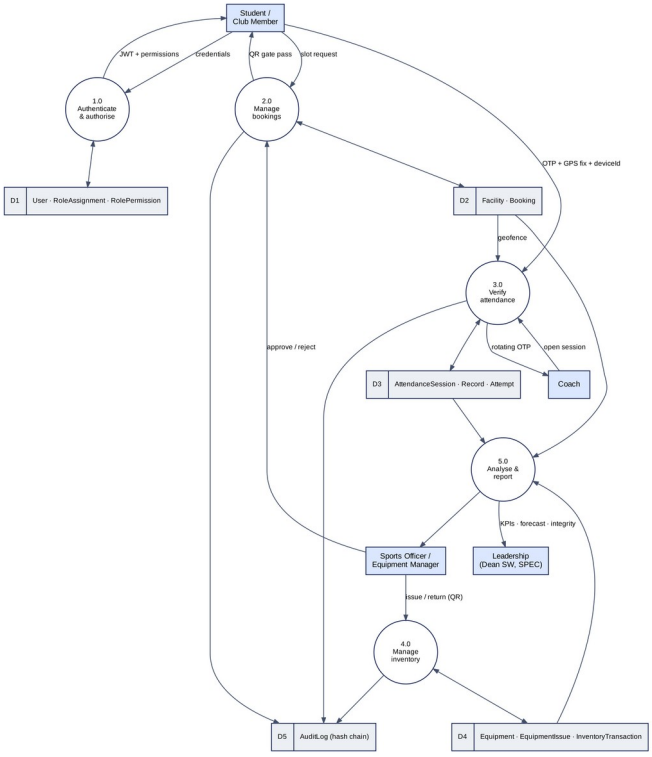


Figure 4.5: *Level-1 data flow diagram*

Table 4.1: *Security controls mapped to ASVS 5.0.0 areas (indicative)*

Area	Implemented control	Gap
Authentication	bcrypt (12 rounds); timing-equalised login; login throttle	No password reset; no staff MFA
Session	12 h JWT; principal reloaded per request	Web token in localStorage
Authorisation	Deny-by-decorator guards; object-level scope checks; spoofed role refused	No penetration test
Validation	Whitelisting DTOs; parameterised queries; upload content sniffing	—
Configuration	helmet headers (CSP, HSTS, nosniff…); CORS allow-list	10 dependency advisories open
Audit	SHA-256 hash-chained AuditLog with /audit/verify	No external log shipping

Deployment (Figure 4.6). `docker-compose.yml` defines three services:

- PostgreSQL 16.4 with a health check and a named volume;
- the API, which runs `prisma migrate deploy` before starting;
- the Next.js server.

The Docker daemon was unavailable during evaluation, so the same components were run natively instead. The container setup is therefore reported as *configured but not verified*. The proposal's AWS/Vercel hosting has not been provisioned.

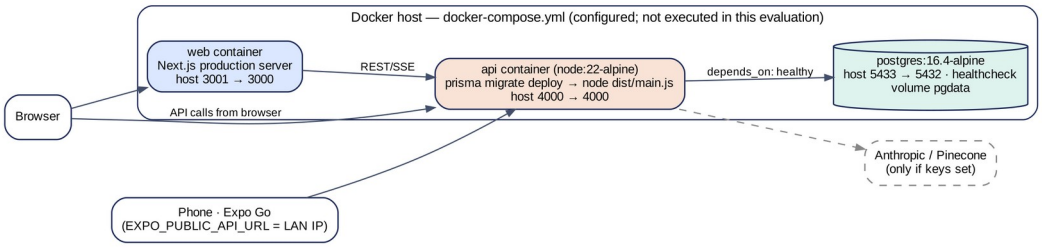


Figure 4.6: *Deployment architecture as configured in docker-compose.yml*

CHAPTER 5

IMPLEMENTATION AND DEVELOPMENT

5.1 Development Environment, Stack and Monorepo

Development and verification used Node.js 22.23, TypeScript 5.9.3, PostgreSQL 16, Jest 29.7 and Playwright 1.63. The repository is an npm-workspaces monorepo with four parts:

- `apps/api` — the NestJS API;
- `apps/web` — the Next.js web client;
- `packages/shared` — shared roles, permissions and types;
- `apps/mobile` — the Expo app, installed separately so that it can pin its own React version.

Together these contain about 28,000 lines of source code (Appendix A).

Table 5.1: *Technology stack (versions verified from package manifests)*

Layer	Technology	Use in SportsOS	Proposal
Web	Next.js 15.5.28, React 19.1.10, Tailwind 4, SWR, Recharts	App Router pages, polling hooks, charts	Next.js 15 ✓
Mobile	Expo SDK 54, React Native 0.81.5	Check-in (location), QR scan (camera), secure token store	RN + Expo ✓
API	NestJS 11.2.7 (jwt, throttler, schedule)	Modules, global guards, cron jobs	NestJS ✓
Data	Prisma 6.19.3, PostgreSQL 16	Typed client, migrations, transactions, advisory locks	Prisma + PostgreSQL ✓
Security	bcryptjs, helmet, class-validator	Hashing, headers, DTO whitelisting	—
AI (optional)	Anthropic SDK 1.1.0, Pinecone 9.1.0	LLM answers, vector index (inactive without keys)	Partly
Infrastructure	Docker Compose	Local container stack	AWS/Vercel not started

Redis is **not** used. Rate-limit counters and the permission cache are held in process memory, which Section 7.3 lists as a limitation.

5.2 Frontend and Backend Implementation

Web client. The web client has 24 authenticated pages. A role-aware dashboard (Figure 5.1) assembles each role's work queues: approvals and live sessions for the Sports Officer, upcoming bookings for students, and the entry log for security staff. Pages that show seeded data carry a visible "DEMO DATA" badge.

API conventions. Controllers declare permissions with decorators. Services enforce ownership, scope and state preconditions, open a transaction for every multi-row change, and call `AuditService.append(tx, …)` inside that transaction. Errors map to precise HTTP status codes: 403 (forbidden), 404 (hidden resource), 409 (state conflict) and 400 (invalid input).

Scheduled jobs. Four jobs close expired sessions, complete or lapse bookings, flag overdue loans and send reminders at 07:00 IST.

Seed data. The deterministic seed creates **synthetic** data: 79 users, 9 facilities, 5 clubs, 2,798 bookings, 1,080 check-ins, 283 loans and 3 tournaments. Every seeded record is flagged `isDemo`, and the seed refuses to wipe a database that contains real users.

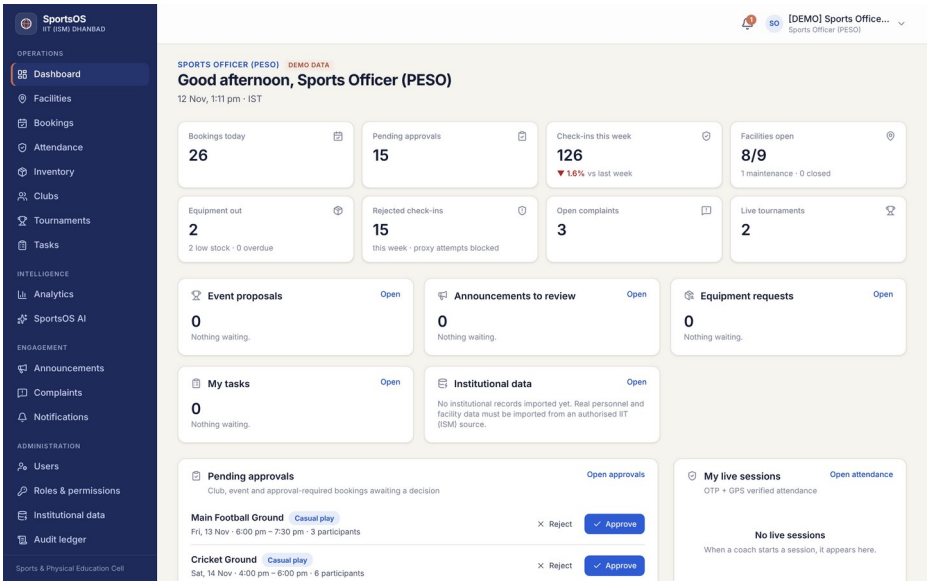


Figure 5.1: *Sports Officer dashboard with KPIs and approval queue (synthetic data)*

5.3 Authentication, User Management and RBAC

Login, registration, `/auth/me` and user administration are implemented. Roles cannot be set through the user-update endpoint; they change only through RBAC assignments. All 19 seeded role accounts signed in during the evaluation. A **forgot/reset-password flow does not exist**: the route returns 404, and building it requires an approved email channel.

RBAC is implemented in three files:

- `access.ts` — pure resolution and scope logic;
- `rbac.service.ts` — principal loading and matrix administration;
- `role-assignments.service.ts` — assignment, two-person approval and revocation, under per-user advisory locks.

Administrators get a live 18 × 70 permission matrix and a roles drawer for scoped and temporary roles (Appendix E, Figure E.3).

5.4 Clubs, Facilities and Booking

Clubs. Clubs have a coach and members with a status and a role. Students request membership, and the coach approves. Coaches manage only the rosters of clubs they coach. In the seed only the Football and Basketball clubs have a coach; Cricket is managed by a sport-scoped coordinator.

Facilities. Each facility stores its opening hours in IST, slot length, capacity, approval flag and geofence. `GET /facilities/:id/availability` returns the day's slot grid (Appendix E, Figure E.4).

Booking (Figure 5.2). A request passes four steps:

1. Validate duration, booking horizon, opening hours, capacity, facility status and booking type.
2. Lock the facility and check for overlapping bookings.
3. Auto-approve eligible casual bookings, or queue the request for an approver.
4. Issue a QR gate pass, which security scans from 15 minutes before the slot.

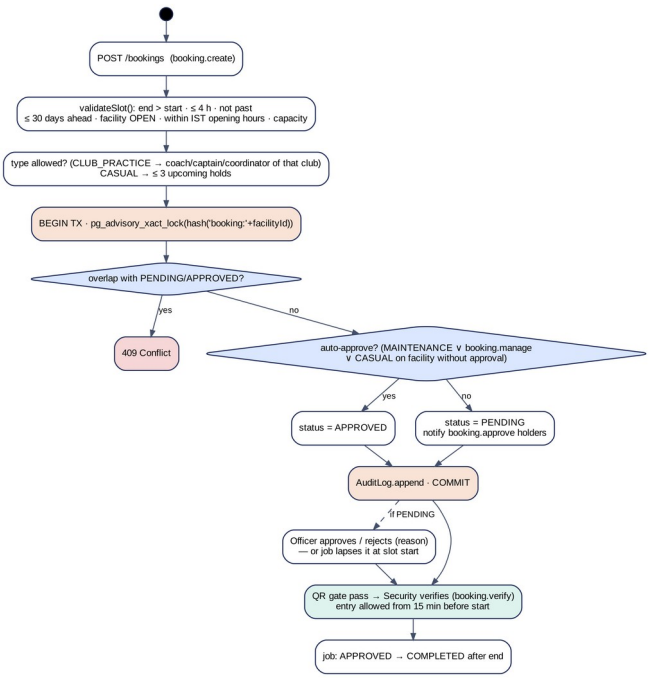


Figure 5.2: *Facility booking workflow (from bookings.service.ts)*

5.5 Verified Attendance

A coach opens a session, and the API stores a random secret that it never returns. The coach's screen shows a 6-digit TOTP that rotates every 30 seconds [20], [21]; the implementation is tested against the RFC test vectors.

The student submits the code, a GPS fix with its accuracy, and a device ID. The server applies its checks in the order of Figure 5.3. The geofence uses the haversine distance [30] with a tolerance of min(accuracy, 25 m). Every attempt is logged, which feeds the integrity analytics, and reuse of a device for a second student alerts the coach. Manual corrections are labelled `MANUAL`, require a reason, and never store fabricated coordinates. Figure 5.4 shows a live session created during the evaluation.

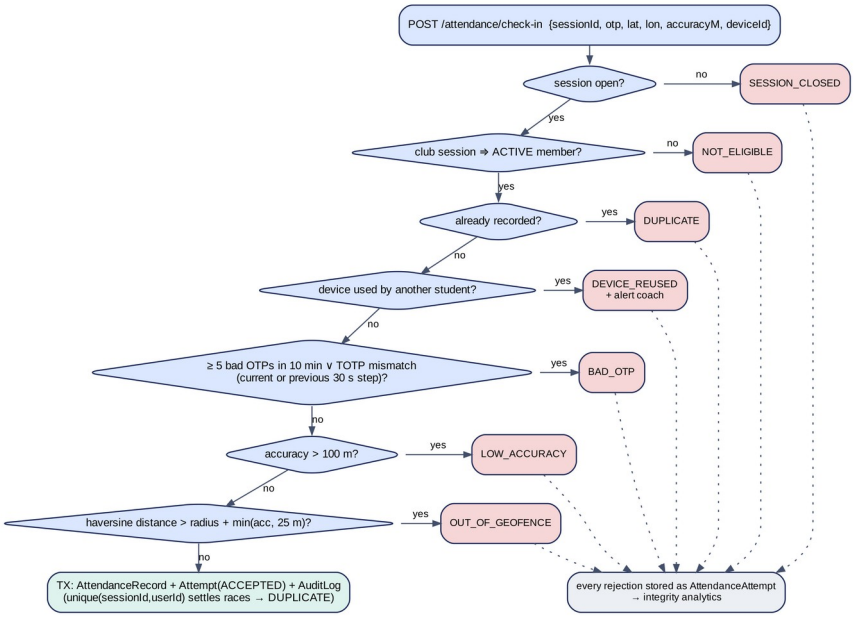


Figure 5.3: *Attendance check-in validation workflow (attendance.service.ts)*

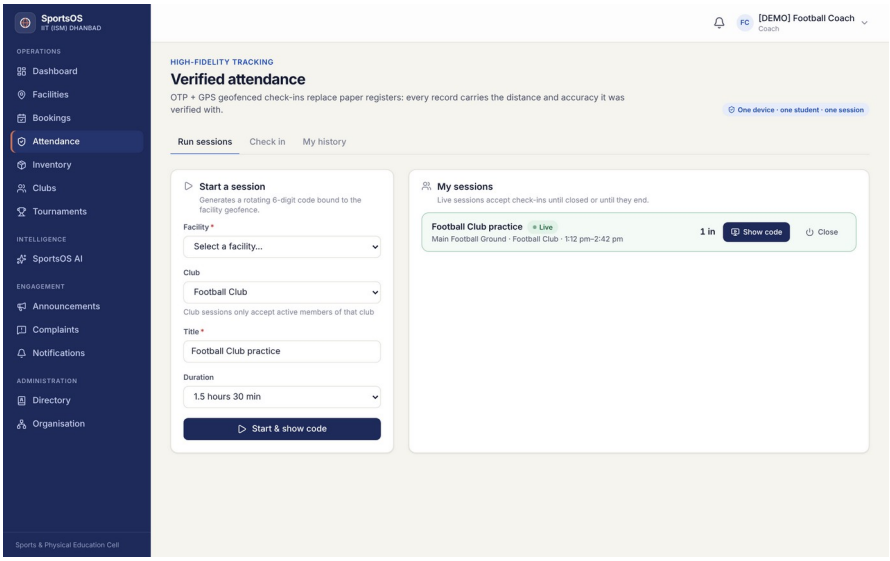


Figure 5.4: *Coach console with a live verified-attendance session*

5.6 Equipment and Inventory

Items carry a QR code and available, damaged and total quantities. Every change goes through `applyStock`, which writes a ledger entry with the deltas and resulting balances. Issue and return follow Figure 5.5: a return records good, damaged and lost units in one transaction. Requests from club members are approved by the equipment manager (never by the requester) and fulfilled through the same ledger. Low-stock and overdue alerts are raised automatically.

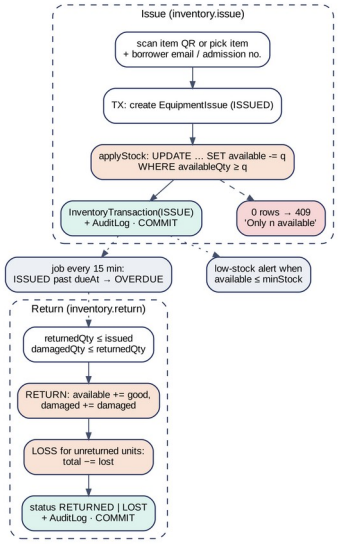


Figure 5.5: *Equipment issue and return workflow*

5.7 Tournaments, Complaints, Notifications and Analytics

Tournaments. Proposals require approval by someone other than the proposer, and hostel secretaries can register only their own hostel's residents. Fixture generation supports two formats: seeded single elimination with byes and automatic advancement, and round robin with 3/1/0 standings (Appendix E, Figure E.2).

Complaints. Complaints move through OPEN → IN_PROGRESS → RESOLVED → CLOSED (Appendix E, Figure E.1). Resolution requires a note, and the raiser cannot resolve their own complaint. Reopening is limited, and escalation brings the complaint to the Dean. Attachments are checked by content sniffing.

Notifications and announcements. Notifications are polled by the client every 30 seconds; there is no push channel yet. Official announcements are published only after approval.

Analytics. The analytics module provides KPIs, participation, utilisation heatmaps, equipment wear, attendance integrity and an additive Holt–Winters demand forecast [31], [32] (Appendix E, Figure E.5). The forecast ran on synthetic history, so its error is not reported as a result.

5.8 Mobile Application and AI Features

Mobile. The Expo app has 11 screens: five tabs (Home, Bookings, Check-in, Scan, Profile) plus booking, equipment and gate-pass screens. It passed a strict TypeScript check but has **not** been run on a device, so its GPS, camera and usability behaviour are unverified.

AI. The assistant implements retrieval-augmented generation [33] over generated documents that contain aggregate figures only, never personal records. Each document is tagged with the permissions required to read it, and documents are filtered by the caller's permissions before retrieval. Without a Pinecone key, retrieval uses in-process BM25 [34]; without a Claude key, the assistant returns the retrieved facts and a templated report. In the evaluation `/ai/status` reported offline mode. Offline retrieval, streaming and permission isolation passed their tests, while the Claude and Pinecone paths were tested only with mocked clients (Appendix E, Figure E.6).

5.9 Module-Wise Implementation Status

Table 5.2: *Module-wise implementation status at mid-evaluation*

Module	Backend	Web	Mobile	Status
Authentication & users	✓	✓	✓	Implemented, tested
Forgot / reset password	✗	✗	✗	Not implemented
RBAC (scoped, time-bound, two-person)	✓	✓	—	Implemented, tested
Facilities, booking, QR gate pass	✓	✓	✓	Implemented, tested
Clubs & rosters	✓	✓	—	Implemented, tested
OTP + GPS attendance	✓	✓	✓	Implemented, tested (API/web)
Inventory ledger & QR	✓	✓	✓	Implemented, tested
Tournaments & fixtures	✓	✓	—	Implemented, tested
Complaints, announcements, tasks	✓	✓	—	Implemented, tested
Notifications	✓	✓	—	Implemented, tested (polling)
Analytics & audit ledger	✓	✓	—	Implemented, tested (synthetic)
Institutional data import	✓	✓	—	Implemented, tested
AI assistant & reports	✓	✓	—	Partial: offline only
Mobile application	—	—	✓	Partial: type-checked
Redis / shared cache	✗	—	—	Not implemented
Cloud deployment	—	—	—	Planned

CHAPTER 6

SOFTWARE TESTING AND EVALUATION

6.1 Testing Strategy

The author obtained all results on **12 November 2026** (Node.js 22.23.4, PostgreSQL 16, Chromium with Playwright 1.63), using production builds and a freshly migrated, synthetically seeded database. The evaluation is **preliminary**: it establishes functional and authorisation correctness, but not performance at scale, field GPS behaviour or usability.

The strategy is layered:

- **unit tests** for security-critical pure logic (OTP, geofence, hash chain, brackets, forecast, import parsing, permission resolution);
- **HTTP tests** against the running API and a real database;
- **browser tests** of user-facing behaviour.

Because broken access control is the leading risk [8], the e2e suite probes 23 protected endpoints with all 18 roles. Allowed roles must not receive a 401 or 403, and denied roles must receive exactly 403.

Table 6.1: *Test levels and results of the 12 November 2026 run*

Level	Tool / command	Result
Static analysis	tsc (shared, API, web, mobile); ESLint	PASS: 0 errors; 2 lint warnings
Schema	prisma validate; migrate deploy	PASS: 2 migrations applied
Unit	Jest, 8 suites	PASS: 71 / 71
API smoke (end to end)	npm run test:smoke	PASS: 83 / 83
RBAC & workflow e2e	npm run test:e2e	PASS: 200 / 200
Browser	Playwright: per role; flows	PASS: 9 / 9; 12 / 12
Targeted checks (author)	Scripted HTTP checks	PASS: 19 logins, 11 attendance cases, restart, port, headers
Container build	docker compose up --build	Not run (no Docker daemon)
Load / penetration / device	—	Not run (planned)

6.2 Unit and Integration Testing

The 71 unit tests (Appendix F, Table F.1) cover:

- the RFC 4226 HOTP vectors;
- geofence verdicts;

- detection of edited and deleted audit entries;
- bracket seeding;
- Holt–Winters fitting;
- CSV import rules;
- expected grants and denials for each of the 18 roles.

Two integration tests exercise the database design directly:

- **Booking race:** six concurrent requests for one slot returned `201, 409, 409, 409, 409, 409`, so exactly one won.
- **Stock race:** eight concurrent single-unit issues against three units in stock returned exactly three 201s. The ledger then reconciled with stock, and the invariant total = available + issued + damaged held.

Both results hold at small scale only.

6.3 Functional, Authentication and Authorisation Testing

Table 6.2 lists representative cases for the scenarios named in the evaluation brief. Appendix F gives all 63 cases across functional, authentication, RBAC, database and attendance testing.

Table 6.2: *Representative test cases (complete tables in Appendix F)*

Test ID	Test scenario	Expected result	Actual result	Status
AUTH-01	Successful login, 19 seeded role accounts	HTTP 200 + token	19 / 19 returned 200	PASS
AUTH-02	Invalid login (unknown account)	HTTP 401	401	PASS
AUTH-04	Unauthorised API access (no token)	HTTP 401	401	PASS
RBAC-01	Role-based route protection: student calls analytics	HTTP 403	403	PASS
RBAC-02	Student spoofs X-Active-Role: SPORTS_OFFICER	HTTP 403	403	PASS
RBAC-15	Role-based navigation in the browser (9 roles)	Permitted items only	9 / 9 as expected	PASS
ATT-01/02	Coach opens a session for own club / another club	201 / 403	201 / 403	PASS
BK-01	Student books a free slot	APPROVED	APPROVED	PASS
BK-04	Booking conflict: 6 concurrent requests	Exactly one succeeds	201 ×1, 409 ×5	PASS
ATT-07/05	Valid check-in / check-in 1.1 km away	ACCEPTED / OUT_OF_GEOFENCE	ACCEPTED (11 m) / OUT_OF_GEOFENCE (1,112 m)	PASS
INV-01/03	Issue by QR; return with one damaged unit	Stock and ledger updated	available 13 → 12, damaged +1	PASS
CMP-02	Assignee resolves; raiser reopens	RESOLVED → OPEN	As expected	PASS
DB-03	Persistence across API restart	Counts unchanged	2,798 bookings; 589 audit entries	PASS

Test ID	Test scenario	Expected result	Actual result	Status
AUTH-09	Forgot-password request	Reset flow	404 (not implemented)	Not impl.

Two issues reported in earlier reviews were rechecked and could not be reproduced.

- **Demo sign-in failures.** The changelog records that the seed previously failed to load `.env`, so the demo accounts were never created; this has since been fixed.
- **Duplicate API processes on port 4000.** A second instance exits immediately with `EADDRINUSE`, so this was most likely a stale development process.

6.4 Database, API, End-to-End and Security Testing

Database and API. The schema validated, migrations applied to an empty database, and the API registered 70 permissions with 346 grants on first start. `/audit/verify` reported a valid chain of 589 entries. The SSE endpoint emitted `sources`, `delta` and `done` events, and unknown body fields were rejected with HTTP 400.

Browser. The per-role test confirmed the expected navigation for nine roles, with zero console errors and zero failed API calls. The 12 flow checks covered the role switcher, the permission matrix, the roles drawer and import preview.

Security. Response headers include CSP, HSTS, X-Frame-Options, nosniff and Referrer-Policy, and origins outside the CORS allow-list receive no `Access-Control-Allow-Origin` header. `npm audit --omit=dev` reports **10 advisories (7 high, 3 moderate)**, which have not yet been triaged. No penetration test or formal ASVS assessment has been performed.

6.5 Preliminary Evaluation

Every executed automated check passed: 71 unit tests, 83 smoke checks, 200 RBAC and workflow checks, 21 browser checks, and the author's targeted checks. The evaluation confirmed two functional gaps (no password reset, no shared cache) and found one cosmetic defect. The defect, **D-01**, is in the attendance form, which labels 90 minutes as "1.5 hours 30 min" because it does not floor the hours.

The results support functional correctness and access-control enforcement at prototype scale. They do not yet support claims about scale, field GPS reliability, mobile usability, AI answer quality or adoption.

CHAPTER 7

RESULTS, LIMITATIONS AND FUTURE WORK

7.1 Current Status and Achievement Against Objectives

Since the August proposal, the project has become a working, tested prototype:

- 31 database models;
- 143 API handlers in 20 modules;
- 28 web routes;
- a type-checked mobile app;
- more than 370 automated checks.

The September–November milestones are met, and the December tournament milestone was delivered early. The January (AI) and February (mobile) milestones are implemented in code but not yet verified.

Table 7.1: *Achievement against the proposal's objectives*

Obj.	Evidence	Assessment
O1	One schema and API for all operational modules	Met (prototype)
O2	Approval workflows, four scheduled jobs, conflict detection, import rollback	Met (prototype)
O3	18 roles, 70 permissions, scoped/time-bound/two-person; 200/200 checks	Met (prototype)
O4	Analytics dashboards and forecast	Partly met: synthetic data
O5	Student web portal and mobile client	Partly met: no user study; mobile untested
O6	Modular monolith, containers, AI with graceful degradation	Partly met: no cloud, load test, live AI or cache

7.2 Demonstration and Engineering Contributions

A demonstration, with every step exercised during the evaluation, runs as follows:

1. A student books a court, and a conflicting request is refused.
2. The Sports Officer approves a pending booking.
3. A coach runs a session; out-of-fence and wrong-code check-ins are rejected and appear in the integrity analytics.
4. Equipment is issued and returned against the ledger.
5. A bracket is shown.

6. A change to the permission matrix takes effect immediately.
7. The audit chain is verified.

The engineering contributions are:

1. scope-aware, time-bounded RBAC without implicit hierarchy;
2. proof-of-presence attendance combining TOTP, accuracy-aware geofencing, device binding and a complete attempt log;
3. database-enforced consistency with a reconciled stock ledger and a hash-chained audit log;
4. a role-exhaustive verification method.

An internal audit of the first version (recorded in the changelog) found privilege escalation by the Sports Officer, cross-club coordinator access, and integrity data leaking to students through the AI assistant. The RBAC overhaul closed all three, and the e2e suite now asserts each fix.

7.3 Current Limitations and Technical Challenges

Table 7.2: *Known limitations at mid-evaluation*

ID	Limitation	Planned action
L-01	No forgot/reset-password flow	Email-token reset once a mail channel is approved
L-02	Web JWT in localStorage (XSS exposure)	httpOnly SameSite cookie session
L-03	No Redis; rate limits and cache are per process	Redis-backed throttler and cache
L-04	10 dependency advisories (7 high)	Triage, upgrade, re-test
L-05	Mobile app not run on devices	Android/iOS device testing
L-06	AI verified offline only	Live evaluation of grounding, refusals, latency
L-07	Synthetic data; placeholder geofences	Ground survey; verified institutional import
L-08	No load, penetration or usability testing; containers not executed	Load tests, ASVS review, usability study, staging

Three challenges shaped the design. **Concurrency without a queue** was solved with facility-keyed advisory locks. **Honest manual corrections** required nullable coordinates and an explicit method field, so that no GPS fix is ever fabricated. **Time zones** needed explicit IST helpers, because opening hours and "today" are defined in IST while timestamps are stored in UTC.

7.4 Future Enhancements and Revised Roadmap

Planned enhancements are:

- password reset, optional two-factor authentication for staff, and cookie sessions;
- Redis for shared rate limiting, caching and push notifications;
- live AI evaluation;
- forecast validation on real booking data against seasonal-naïve baselines [32];
- institute SSO, subject to approval;
- an accessibility review.

Because the core modules finished early, the remaining months are re-planned towards validation and hardening (Figure 7.1):

- **November–January:** SPEC validation and user-acceptance testing; ground survey;
- **December–January:** security hardening;
- **January–February:** AI evaluation;
- **February:** mobile device testing;
- **March–April:** load and penetration tests, staging deployment and the thesis.

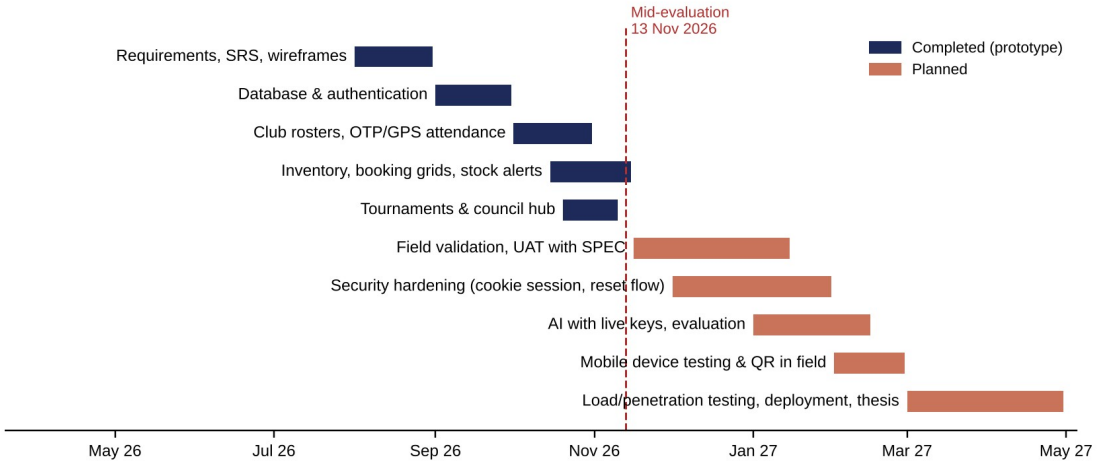


Figure 7.1: *Revised development roadmap*

7.5 Conclusion

At mid-evaluation, SportsOS is a working prototype that unifies booking, verified attendance, inventory, tournaments, complaints and analytics for eighteen stakeholder roles. Tests executed for this report verify its central claims: access is confined to each role's permissions and scopes; attendance requires physical presence; bookings and stock stay consistent under concurrency; and every state change is captured in a verifiable audit chain.

The limitations are equally clear: no password reset, no shared cache, an untested mobile client, offline-only AI verification, synthetic data, and no load, penetration or user testing.

The second half of the project is devoted to stakeholder validation and hardening for deployment at IIT (ISM) Dhanbad.

REFERENCES

[1] A. Kumar, "SportsOS: An AI-Powered Sports ERP for IIT (ISM) Dhanbad," Integrated M.Tech thesis proposal, Dept. of Mathematics and Computing, IIT (ISM) Dhanbad, Aug. 2026.

[2] Sports & Physical Education Center (SPEC), IIT (ISM) Dhanbad, "Activities." [Online]. Available: https://people.iitism.ac.in/~dsw/spec/activity.html (content consulted through a search-engine index on 12 Nov. 2026; page undated).

[3] Sports & Physical Education Center (SPEC), IIT (ISM) Dhanbad, "Facilities." [Online]. Available: https://people.iitism.ac.in/~dsw/spec/facilities.html (content consulted through a search-engine index on 12 Nov. 2026; page undated).

[4] Indian Institute of Technology (Indian School of Mines) Dhanbad, "SPEC." [Online]. Available: https://www.iitism.ac.in/spec (host not reachable from the evaluation environment on 12 Nov. 2026).

[5] Indian Institute of Technology (Indian School of Mines) Dhanbad, "Facilities — Dean (Students' Welfare)." [Online]. Available: https://www.iitism.ac.in/facilities-dsw (host not reachable from the evaluation environment on 12 Nov. 2026).

[6] L. Haffner, I. Oshri, and J. Kotlarsky, "Directions for future IS research on sports digitalisation: A stakeholder perspective," *The Journal of Strategic Information Systems*, vol. 34, no. 2, Art. no. 101905, 2025, doi: 10.1016/j.jsis.2025.101905.

[7] D. E. W. Saputra and P. Asmawati, "Elevating user experience and service excellence in sports facilities via integrated management information systems: A systematic review," *Sports Science and Health*, vol. 15, no. V, pp. 111–118, 2025, doi: 10.7251/SSH25V111S.

[8] OWASP Foundation, *OWASP Top 10: 2021*. [Online]. Available: https://owasp.org/Top10/

[9] OWASP Foundation, *Application Security Verification Standard (ASVS)*, version 5.0.0, May 2025. [Online]. Available: https://owasp.org/www-project-application-security-verification-standard/

[10] G. Vial, "Understanding digital transformation: A review and a research agenda," *The Journal of Strategic Information Systems*, vol. 28, no. 2, pp. 118–144, 2019.

[11] M. D. Shahrom and K. A. Mohamad Sukri, "Swimming facilities management system for Universiti Tun Hussein Onn Malaysia Sport Centre with two-factor authentication," *Applied Information Technology and Computer Science*, vol. 6, no. 2, pp. 625–643, 2025.

[12] The PostgreSQL Global Development Group, "Explicit locking — Advisory locks," *PostgreSQL 16 Documentation*, §13.3.5. [Online]. Available: https://www.postgresql.org/docs/16/explicit-locking.html

[13] T. H. Davenport, "Putting the enterprise into the enterprise system," *Harvard Business Review*, vol. 76, no. 4, pp. 121–131, 1998.

[14] H. Klaus, M. Rosemann, and G. G. Gable, "What is ERP?," *Information Systems Frontiers*, vol. 2, no. 2, pp. 141–162, 2000.

[15] R. E. Freeman, *Strategic Management: A Stakeholder Approach*. Boston, MA, USA: Pitman, 1984.

[16] R. Sandhu, D. Ferraiolo, and R. Kuhn, "The NIST model for role-based access control: Towards a unified standard," in *Proc. 5th ACM Workshop on Role-Based Access Control (RBAC '00)*, Berlin, Germany, 2000, pp. 47–63, doi: 10.1145/344287.344301.

[17] D. F. Ferraiolo, R. Sandhu, S. Gavrila, D. R. Kuhn, and R. Chandramouli, "Proposed NIST standard for role-based access control," *ACM Transactions on Information and System Security*, vol. 4, no. 3, pp. 224–274, 2001.

[18] N. Provos and D. Mazières, "A future-adaptable password scheme," in *Proc. USENIX Annual Technical Conference, FREENIX Track*, Monterey, CA, USA, 1999, pp. 81–91.

[19] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," IETF RFC 7519, May 2015.

[20] D. M'Raihi, M. Bellare, F. Hoornaert, D. Naccache, and O. Ranen, "HOTP: An HMAC-based one-time password algorithm," IETF RFC 4226, Dec. 2005.

[21] D. M'Raihi, S. Machani, M. Pei, and J. Rydell, "TOTP: Time-based one-time password algorithm," IETF RFC 6238, May 2011.

[22] *Systems and Software Engineering — Life Cycle Processes — Requirements Engineering*, ISO/IEC/IEEE 29148:2018, 2018.

[23] *Systems and Software Engineering — Systems and Software Quality Requirements and Evaluation (SQuaRE) — Product Quality Model*, ISO/IEC 25010:2023, 2023.

[24] M. Cohn, *User Stories Applied: For Agile Software Development*. Boston, MA, USA: Addison-Wesley, 2004.

[25] I. Sommerville, *Software Engineering*, 10th ed. Harlow, U.K.: Pearson, 2016.

[26] A. R. Hevner, S. T. March, J. Park, and S. Ram, "Design science in information systems research," *MIS Quarterly*, vol. 28, no. 1, pp. 75–105, 2004.

[27] L. Bass, P. Clements, and R. Kazman, *Software Architecture in Practice*, 4th ed. Boston, MA, USA: Addison-Wesley, 2021.

[28] Vercel, Inc., *Next.js Documentation — App Router*. [Online]. Available: https://nextjs.org/docs/app

[29] NestJS, "Guards," *NestJS Documentation*. [Online]. Available: https://docs.nestjs.com/guards

[30] R. W. Sinnott, "Virtues of the haversine," *Sky and Telescope*, vol. 68, no. 2, p. 159, 1984.

[31] P. R. Winters, "Forecasting sales by exponentially weighted moving averages," *Management Science*, vol. 6, no. 3, pp. 324–342, 1960.

[32] R. J. Hyndman and G. Athanasopoulos, *Forecasting: Principles and Practice*, 3rd ed. Melbourne, Australia: OTexts, 2021.

[33] P. Lewis *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, 2020, pp. 9459–9474.

[34] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.

APPENDIX A: REPOSITORY STRUCTURE

Table A.1: *Source size by area (lines, measured with wc -l on .ts/.tsx/.mjs/.cjs/.prisma/.sql files)*

Area	Path	Lines
API source	apps/api/src	7,631
Schema, migrations, seed	apps/api/prisma	2,566
API end-to-end tests	apps/api/test	600
Web pages	apps/web/app	7,013
Web components and library	apps/web/components, lib	5,734
Web browser tests	apps/web/test	146
Mobile	apps/mobile/app, src	3,245
Shared package	packages/shared/src	1,034
Total		27,969

Top-level structure of the repository as inspected (dependency and build directories omitted).

```
sportsos/
├── apps/
│   ├── api/                  NestJS 11 API
│   │   ├── prisma/           schema.prisma, migrations/ (2), seed.ts, bootstrap-admin.ts
│   │   ├── src/              achievements · ai · analytics · announcements · attendance ·
audit
│   │   │                     auth · bookings · clubs · common · complaints · facilities ·
health
│   │   │                     institution · inventory · jobs · notifications · prisma ·
rbac
│   │   │                     tasks · tournaments · users · app.module.ts · main.ts
│   │   ├── test/             smoke.mjs (83 checks) · rbac.e2e.mjs (200 checks)
│   │   └── Dockerfile
│   ├── web/                  Next.js 15 client
│   │   ├── app/(app)/        24 authenticated pages · login · register
│   │   ├── components/       shell · charts · attendance projector · qr-scanner · ui
│   │   ├── lib/              api.ts · hooks.ts · speech.ts · format.ts
│   │   ├── test/             ui-roles.e2e.cjs · ui-flows.e2e.cjs
│   │   └── Dockerfile
│   └── mobile/               Expo SDK 54 app (separate install)
│       ├── app/              (tabs)/ index · bookings · checkin · scan · profile
│       │                     book/ · booking/[id] · equipment/[code] · pass/[token] ·
login
│       └── src/              api.ts · auth.tsx · device.ts · qr.ts · hooks/
├── packages/shared/src/      roles.ts · permissions.ts · types.ts
├── docs/                     API, RBAC matrix, data integration, audits, test report
└── docker-compose.yml
```

APPENDIX B: SELECTED API DOCUMENTATION

Table B.1: *API surface by module (counted from controller decorators)*

Module	Base path	Handlers	Representative endpoints
Auth	/auth	3	POST login, POST register, GET me
Users / RBAC	/users, /rbac	4 + 10	GET/PUT/DELETE rbac/matrix, POST rbac/assignments, …/approve, …/revoke
Facilities	/facilities	6	GET :id/availability, PATCH :id/status
Bookings	/bookings	10	POST, POST check-in, POST :id/approve|reject|cancel|reschedule, GET :id/pass
Clubs	/clubs	10	POST :id/join, PATCH :id/members/:mid, POST :id/announcements
Attendance	/attendance	10	POST sessions, GET sessions/:id/otp, POST check-in, POST sessions/:id/corrections
Inventory	/inventory	23	GET items/by-qr/:code, POST issues, POST issues/:id/return, GET alerts, requests/*
Tournaments	/tournaments, /matches	13	POST :id/approve, POST :id/teams, POST :id/generate, PATCH matches/:id
Complaints	/complaints	9	PATCH :id, POST :id/escalate|close|reopen|attachments
Announcements / Tasks	/announcements, /tasks	7 + 3	POST :id/submit|approve|publish; PATCH tasks/:id/status
Notifications	/notifications	5	GET unread-count, POST read-all, POST broadcast
Analytics	/analytics	9	overview, participation, utilization, forecast, equipment, integrity, export
AI	/ai	6	GET status, POST ask, POST ask/stream (SSE), POST report
Audit / Institution / Other	various	2 + 9 + 4	GET audit/verify, POST institution/imports/:id/commit|rollback, GET health

All paths are prefixed with `/api`. "Perm." is the permission checked by the route guard; further object-level scope checks apply in the service.

Table B.2: *Selected endpoints*

Method & path	Perm.	Request (key fields)	Responses
POST /auth/login	public	email, password	200 {accessToken, user, permissions, heldRoles}; 401
GET /auth/me	any authenticated	—	200 session view
GET /facilities/:id/availability	facility.view	date (YYYY-MM-DD)	200 slot grid
POST /bookings	booking.create	facilityId, start, end, purpose, type, participants?, clubId?	201 booking (APPROVED|PENDING); 400; 409 overlap
POST /bookings/:id/approve	booking.approve	note?	200; 409 if not PENDING
POST /bookings/check-in	booking.verify	pass token (QR)	200 entry recorded; 409 too early/late
POST /attendance/sessions	attendance.manage (scoped)	title, facilityId, clubId?, endsAt	201 session (secret omitted); 403 out of scope
GET /attendance/sessions/:id/otp	attendance.manage (scoped)	—	200 {code, step, expiresInSec}

Method & path	Perm.	Request (key fields)	Responses
POST /attendance/check-in	attendance.mark	sessionId, otp, latitude, longitude, accuracyM, deviceId?	201 {outcome, message, distanceM, record}
POST /inventory/issues	inventory.issue	equipmentId | qrCode, borrower, quantity, dueAt?	201 issue; 404; 409 insufficient stock
POST /inventory/issues/:id/return	inventory.return	returnedQty, damagedQty?, note?	200 RETURNED|LOST
POST /tournaments/:id/generate	event.publish	—	200 matches (bracket or round robin)
PATCH /complaints/:id	complaint.* (role-dependent)	status?, assignedToId?, resolution?	200; 400 resolution required; 403
PUT /rbac/matrix/:role/:permission	role.manage_permissions	—	200 role permissions
POST /rbac/assignments	role.assign	userId, role, scopeType?, scopeId?, validUntil?, reason	201 ACTIVE | PENDING_APPROVAL
GET /audit/verify	audit.view	—	200 {valid, brokenAtSeq, checked, headHash}
POST /ai/ask/stream	ai.use	question	SSE: sources, delta…, done

APPENDIX C: DATABASE ENTITY CATALOGUE

Table C.1: *All 31 Prisma models grouped by domain*

Domain	Models
Identity & access	User, RoleAssignment, RolePermission, PermissionDefinition
Facilities & bookings	Facility, Booking
Clubs	Club, ClubMembership
Verified attendance	AttendanceSession, AttendanceRecord, AttendanceAttempt, AttendanceCorrection
Inventory	Equipment, EquipmentIssue, InventoryTransaction, EquipmentRequest
Tournaments	Tournament, Team, TeamMember, Match
Engagement	Notification, Complaint, ComplaintAttachment, Announcement, OperationalTask, Achievement
Governance & data	AuditLog, AiUsageLog, InstitutionalRecord, ImportBatch, ImportBatchItem

Table C.2: *Seeded synthetic data volumes (run of 12 Nov 2026)*

Entity	Rows	Entity	Rows
User	79	AttendanceSession	120
RoleAssignment	156	AttendanceRecord	1,080
Facility	9	Equipment	15
Club	5	EquipmentIssue	283
Booking	2,798	Tournament	3
Complaint	5	Notification	83
RolePermission	346	AuditLog	588 (589 after a test action)

APPENDIX D: IMPLEMENTATION EVIDENCE MATRIX

Each requirement is traced to the code that implements it and the test that verified it on 12 November 2026.

Table D.1: *Requirement → implementation → verification evidence*

Req.	Implementation (file)	Verification evidence	Status
FR-01	auth/auth.service.ts, auth.controller.ts	AUTH-01…08; smoke login, register	Verified
FR-02	— (no route)	AUTH-09 → 404	Not implemented
FR-03	rbac/role-assignments.service.ts	RBAC-10…13; e2e role-assignment safeguards (24)	Verified
FR-04	rbac/rbac.service.ts (setGrant, resetRole)	RBAC-14; e2e matrix grant/revoke; UI flow	Verified
FR-05	facilities/facilities.service.ts	smoke availability grid; e2e scoped ground staff	Verified
FR-06	bookings/bookings.service.ts, common/qr.ts	BK-01…08	Verified
FR-07	attendance/*, common/otp.ts, common/geo.ts	ATT-01…12; otp/geo unit tests	Verified
FR-08	inventory/inventory.service.ts (applyStock); jobs	INV-01…06; smoke alerts	Verified (job timing not tested)
FR-09	tournaments/*, bracket.ts	TRN-01, TRN-02; bracket unit tests; RBAC-08	Verified
FR-10	complaints/complaints.service.ts	CMP-01…05; e2e complaints (16)	Verified
FR-11	notifications, announcements modules	smoke notifications, broadcast; e2e announcements	Verified
FR-12	analytics/analytics.service.ts, forecast.ts	smoke analytics (6); forecast unit tests	Verified (synthetic data)
FR-13	ai/ai.service.ts, knowledge.service.ts, vector-store.ts	smoke AI (5); e2e AI isolation (5); unit (8, mocked LLM)	Partial
FR-14	apps/mobile/*	tsc --noEmit exit 0	Partial
FR-15	audit/audit.service.ts, common/hash-chain.ts	DB-04; hash-chain unit tests	Verified
NFR-03	advisory locks; conditional UPDATE	BK-04, INV-04 (small-scale concurrency)	Partly verified
NFR-06	docker-compose.yml, Dockerfiles	not executed	Configured
NFR-07	—	not tested	Not tested

APPENDIX E: ADDITIONAL SCREENSHOTS

All screenshots were captured from the production web build on 12 November 2026 using synthetic demo data.

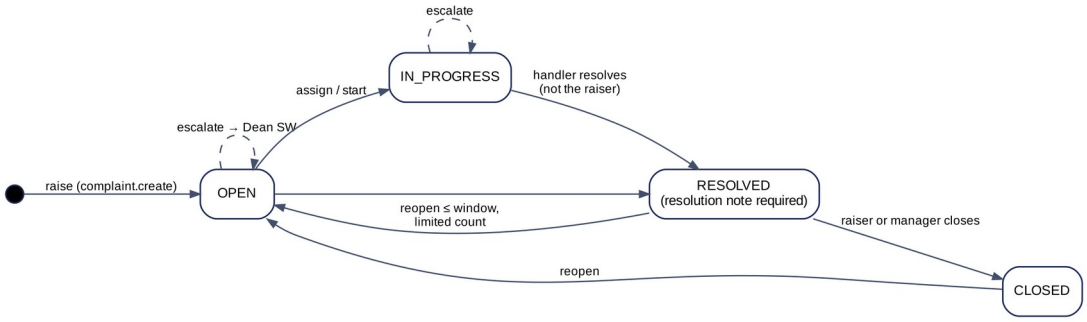


Figure E.1: *Complaint lifecycle state machine (complaints.service.ts)*

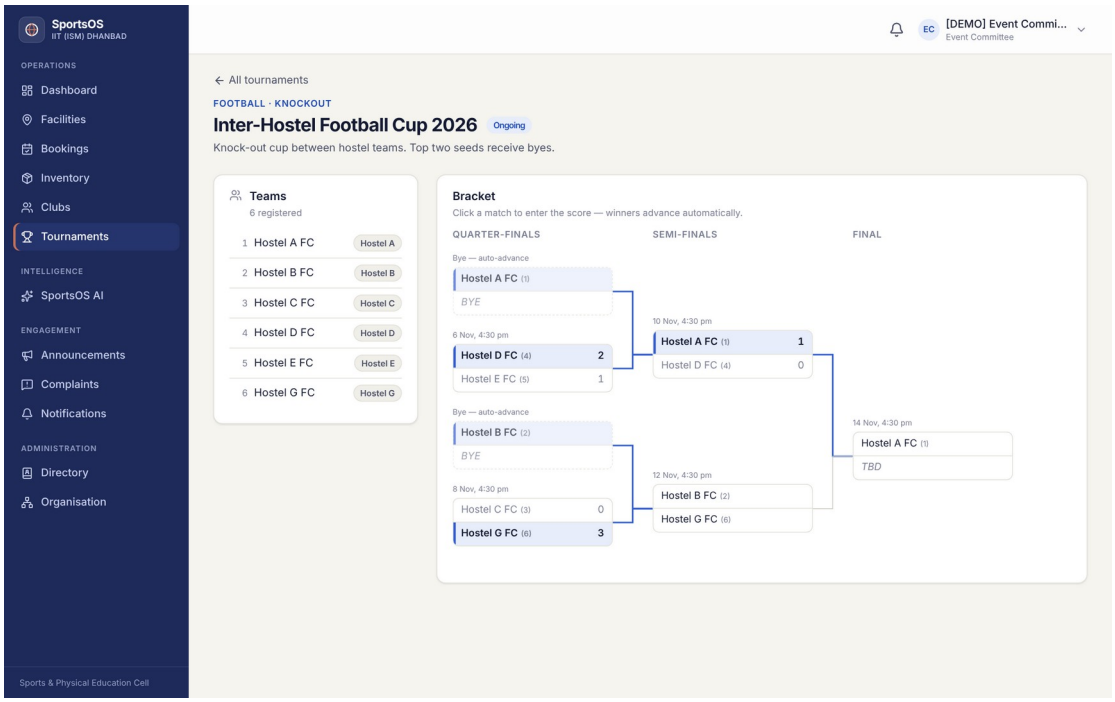


Figure E.2: *Knock-out bracket with seeded byes and automatic advancement (synthetic data)*

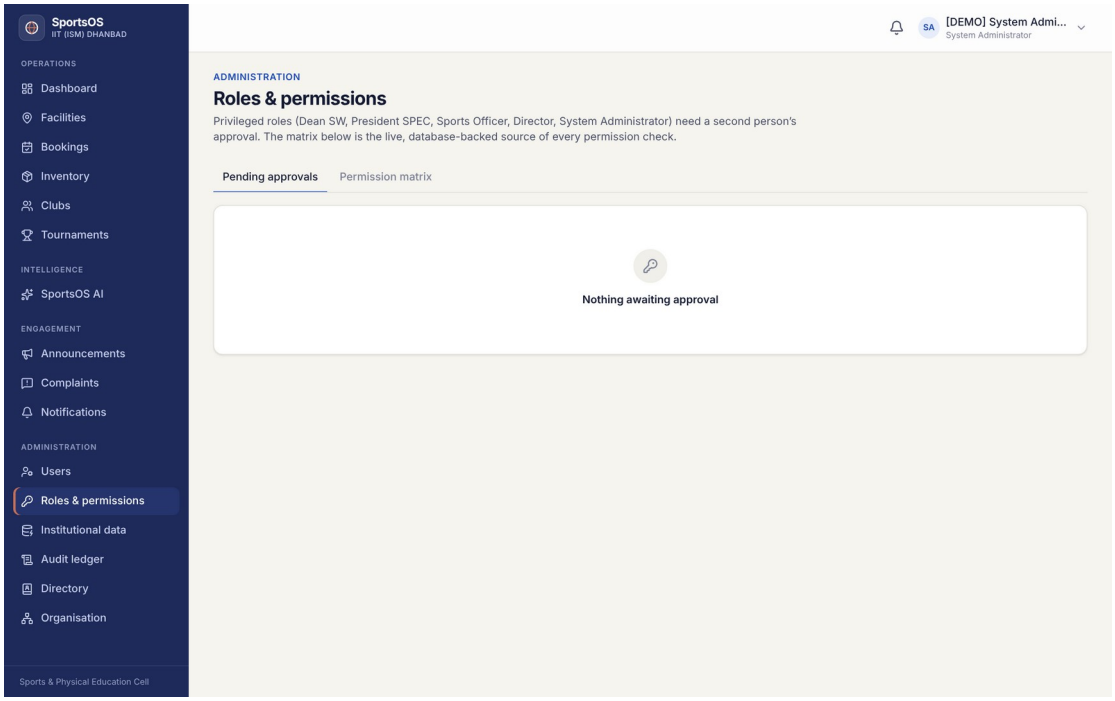


Figure E.3: *Administrator view of roles and the editable role × permission matrix (synthetic data)*

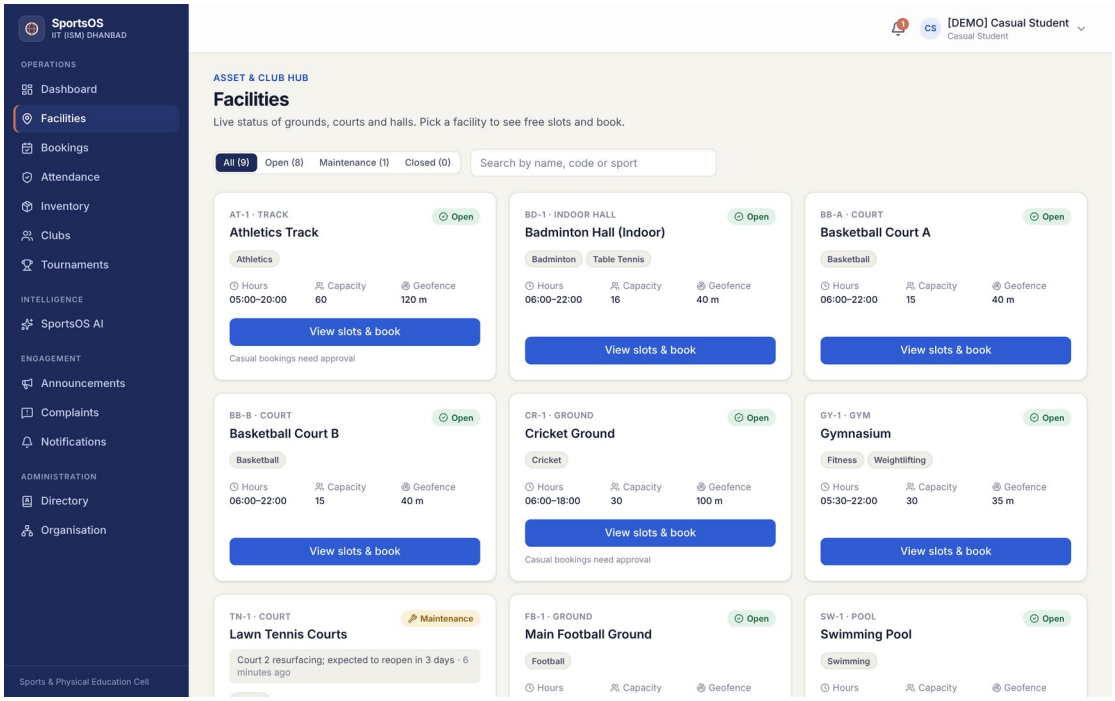


Figure E.4: *Facility page with the daily availability grid (synthetic data)*

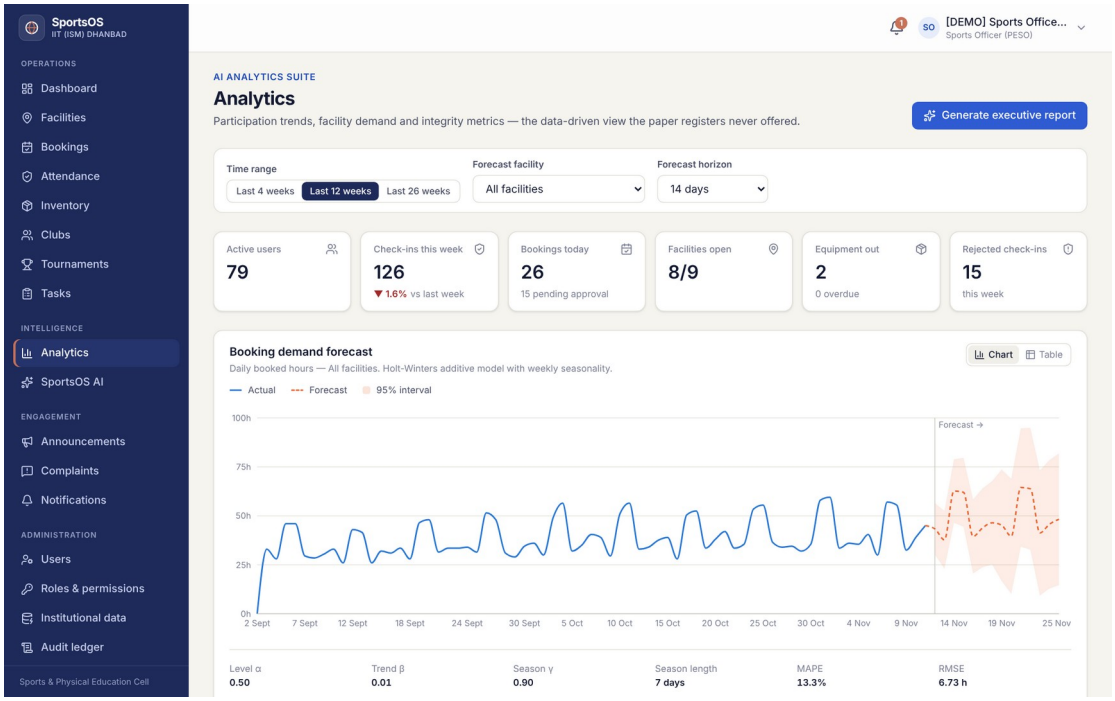


Figure E.5: *Analytics page: participation, utilisation and forecast views (synthetic data)*

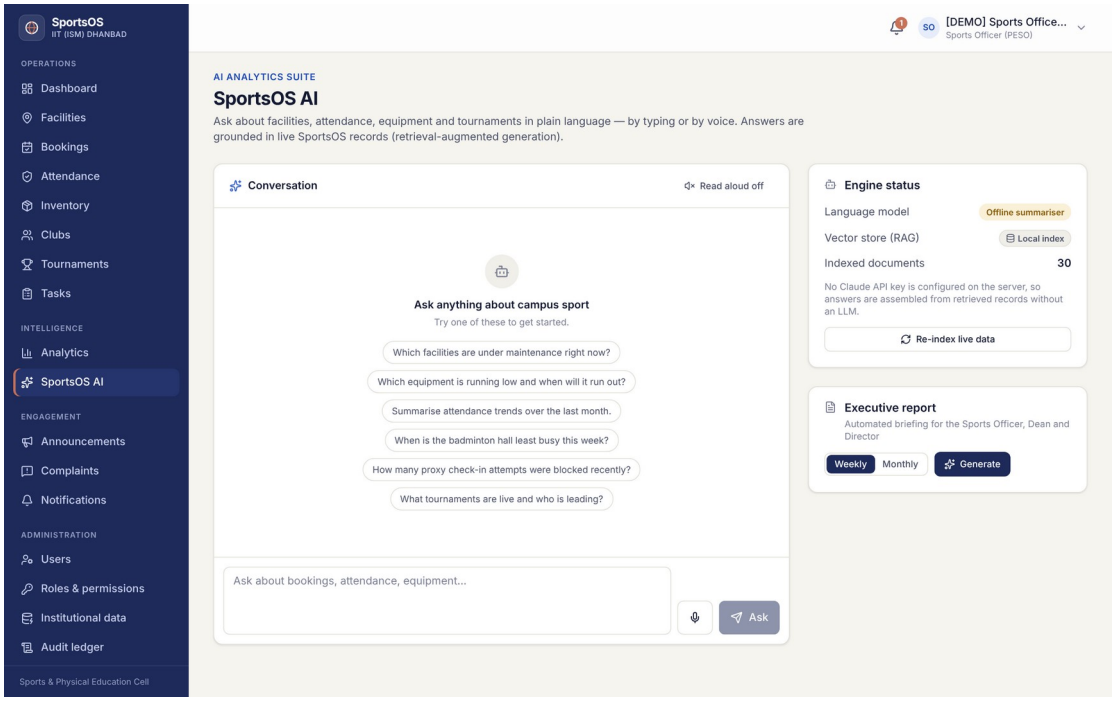


Figure E.6: *SportsOS AI assistant page (offline retrieval mode) (synthetic data)*

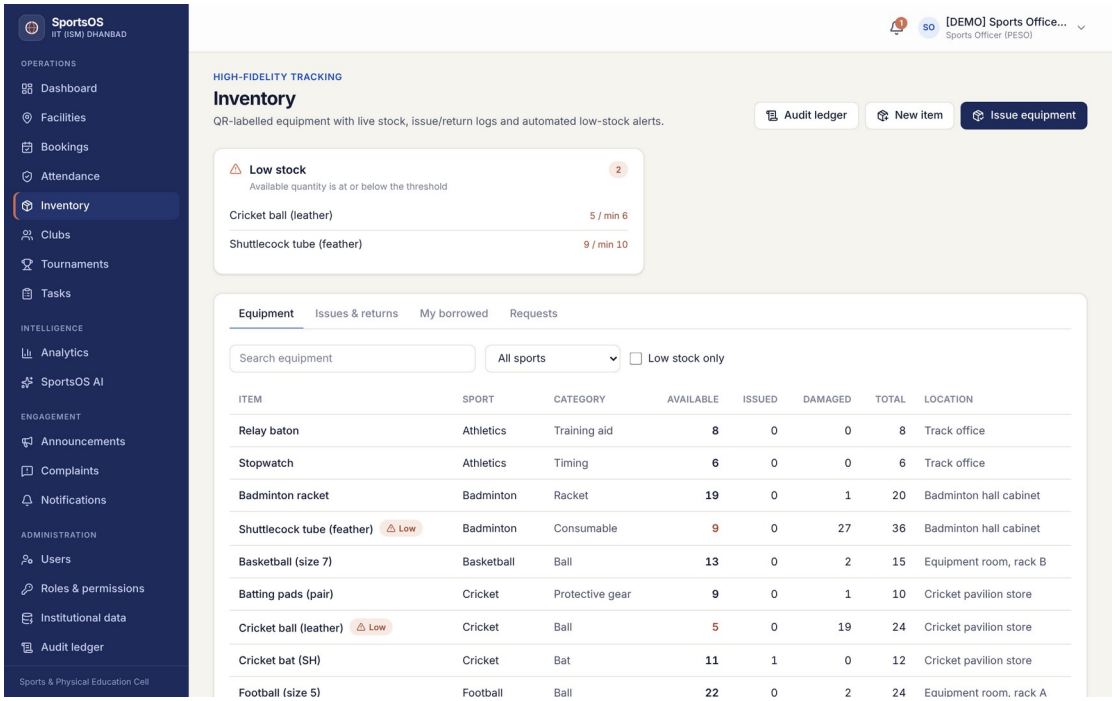


Figure E.7: *Inventory view with stock levels and low-stock alerts (synthetic data)*

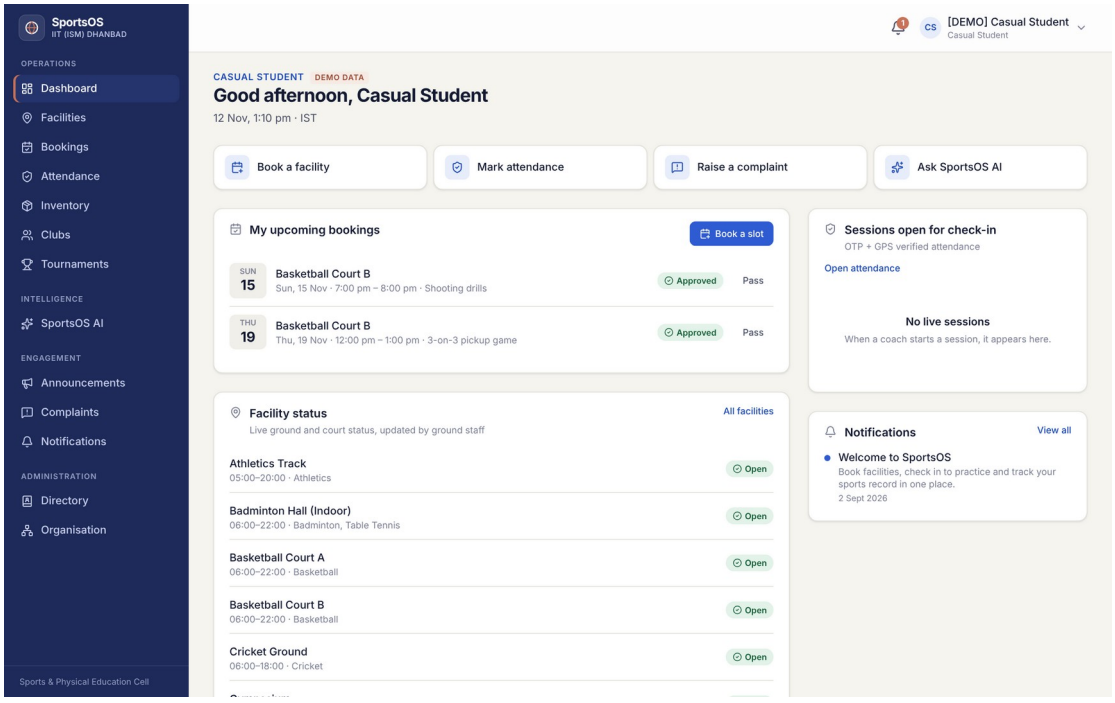


Figure E.8: *Student dashboard (synthetic data)*

APPENDIX F: DETAILED TEST CASES

Complete test-case tables from the verification run of 12 November 2026. BK, INV, CMP and TRN cases come from the API smoke and RBAC/workflow e2e suites; AUTH-01, AUTH-09, ATT-01 to ATT-11 and the DB cases were executed directly by the author.

Table F.1: *Unit test suites (Jest), 71 tests, all passing*

Suite	Tests	What is verified
rbac/access.spec.ts	32	Union of roles; revoked, pending, expired and future assignments ignored; active-role narrowing; spoofed active role rejected; no seniority inheritance; coach/sport/hostel/facility scopes; expected grants and denials for each of the 18 roles
institution/import-parser.spec.ts	11	RFC 4180 CSV parsing (quotes, CRLF, BOM), required columns, provenance rules, duplicate keys
ai/ai.service.spec.ts	8	Offline answers; mocked Claude streaming, refusal and fallback; permission isolation of retrieved documents
tournaments/bracket.spec.ts	5	Seeding and byes, advancement links, round-robin schedule and standings
common/otp.spec.ts	4	RFC 4226 HOTP test vectors; TOTP current/previous-step acceptance
common/geo.spec.ts	4	Haversine distance; inside, outside and low-accuracy verdicts
common/hash-chain.spec.ts	4	Canonical JSON; chain verification; detection of edited and deleted entries
analytics/forecast.spec.ts	3	Holt–Winters fitting and prediction-interval behaviour

Table F.2: *Functional test cases: booking, inventory and complaints (results from the smoke and e2e runs)*

Test ID	Test scenario	Expected result	Actual result	Status
BK-01	Student books a free slot on a facility that needs no approval	Booking APPROVED	APPROVED	PASS
BK-02	Overlapping booking for the same facility	HTTP 409	409 "Slot overlaps an existing booking"	PASS
BK-03	Booking outside opening hours	HTTP 400	400 "open 06:00–22:00 IST"	PASS
BK-04	6 concurrent requests for one slot	Exactly one succeeds	201 ×1, 409 ×5	PASS
BK-05	Booking on a facility that needs approval	PENDING; officer can approve	PENDING → APPROVED	PASS
BK-06	Approving an already approved booking	HTTP 409	409	PASS
BK-07	Gate check-in more than 15 min early	HTTP 409	409 "Too early"	PASS
BK-08	Student opens another student's pass	HTTP 404	404	PASS
INV-01	Issue item by QR to a borrower identified by admission no.	Issue created, stock −1	Created; stock decremented	PASS
INV-02	Issue more than available	HTTP 409	409 "Only 11 × Basketball (size 7) available"	PASS
INV-03	Return with damaged quantity	available/damaged updated	available 13 → 12, damaged +1	PASS
INV-04	8 concurrent issues on 3 units	Exactly 3 succeed	201 ×3, 409 ×5	PASS
INV-05	Ledger deltas vs stock after all movements	Reconcile exactly	Reconciled (10 txns)	PASS
INV-06	Requester approves own equipment request	Refused	Refused	PASS

Test ID	Test scenario	Expected result	Actual result	Status
CMP-01	Resolve complaint without a note	HTTP 400	400	PASS
CMP-02	Assignee resolves; raiser reopens	RESOLVED → OPEN	As expected	PASS
CMP-03	Raiser tries to escalate	Refused	Refused	PASS
CMP-04	Officer escalates; Dean views queue	Visible to Dean	Visible	PASS
CMP-05	Upload script disguised as an image	Rejected	Rejected by content sniffing	PASS
TRN-01	Generate knock-out bracket; play to champion	Byes, advancement, champion	7 matches; champion set	PASS
TRN-02	Event Committee approves its own proposal	Refused	Refused	PASS

Table F.3: *Authentication test cases*

Test ID	Test scenario	Expected result	Actual result	Status
AUTH-01	Sign in with each of the 19 seeded role accounts	HTTP 200 + token for all	19 / 19 returned 200 with role and permissions	PASS
AUTH-02	Sign in with an unknown account	HTTP 401	401	PASS
AUTH-03	Login response contains the password hash	Never	Absent	PASS
AUTH-04	Protected route without a token	HTTP 401	401	PASS
AUTH-05	Token with a tampered signature	HTTP 401	401	PASS
AUTH-06	Register with a non-institute email	HTTP 400	400	PASS
AUTH-07	Register with an institute email	Account with STUDENT role	STUDENT	PASS
AUTH-08	Use a token after the account is deactivated	Rejected	Rejected	PASS
AUTH-09	POST /auth/forgot-password	Reset flow	404: endpoint does not exist	Not impl.

Table F.4: *Authorisation and RBAC test cases*

Test ID	Test scenario	Expected result	Actual result	Status
RBAC-01	Student calls GET /analytics/overview	HTTP 403	403	PASS
RBAC-02	Student sends X-Active-Role: SPORTS_OFFICER	HTTP 403 (not ignored)	403	PASS
RBAC-03	Unknown body field (mass assignment)	HTTP 400	400	PASS
RBAC-04	Football coach opens a session for the Basketball club	HTTP 403	403 (author's check ATT-02)	PASS
RBAC-05	Football coach opens a session for the Football club	HTTP 201	201 (ATT-01)	PASS
RBAC-06	Basketball coach reads the Football session OTP	Refused	403 (ATT-03)	PASS
RBAC-07	Coach edits another club's roster / reads its report	Refused	Refused	PASS
RBAC-08	Hostel A secretary registers a team for another hostel	Refused	Refused	PASS
RBAC-09	Facility-scoped ground staff changes another facility	Refused	Refused	PASS
RBAC-10	Sports Officer or Dean grants ADMIN	Refused	Refused	PASS
RBAC-11	Privileged role assigned → effective?	PENDING until a second person approves	PENDING → approved by admin → effective	PASS
RBAC-12	Revoke a role; reuse the same token	Revocation immediate	Immediate	PASS

Test ID	Test scenario	Expected result	Actual result	Status
RBAC-13	Temporary role past validUntil (before expiry job runs)	No longer effective	Not effective	PASS
RBAC-14	Remove matrix-management permission from ADMIN	Refused (lock-out protection)	Refused	PASS
RBAC-15	Per-role navigation in the browser (9 roles)	Expected menu items only	9 / 9 as expected; 0 console errors	PASS

Table F.5: *Database test cases (author's checks)*

Test ID	Test scenario	Expected result	Actual result	Status
DB-01	Validate schema; apply migrations to an empty database	Valid; migrations applied	Valid; 2 migrations applied	PASS
DB-02	Permission bootstrap on first API start	70 definitions, 346 default grants	PermissionDefinition = 70; RolePermission = 346	PASS
DB-03	Persistence across API restart	Row counts unchanged	Bookings 2,798 → 2,798; AuditLog 589 → 589	PASS
DB-04	Audit chain verification (GET /audit/verify)	valid = true	valid; 589 entries checked; brokenAtSeq = null	PASS
DB-05	Duplicate attendance record for one student	Prevented by unique index	DUPLICATE (ATT-08)	PASS
DB-06	Stock invariant after concurrent issues	total = available + issued + damaged	Holds	PASS

Table F.6: *Attendance verification test cases (author's run against a live session)*

Test ID	Test scenario	Expected result	Actual result	Status
ATT-01	Coach opens a session for own club	HTTP 201	201	PASS
ATT-02	Coach opens a session for a club they do not coach	HTTP 403	403	PASS
ATT-03	Other coach reads the session OTP	Refused	403	PASS
ATT-04	Member submits a wrong OTP	BAD_OTP	BAD_OTP	PASS
ATT-05	Correct OTP, position ≈ 1.1 km away	OUT_OF_GEOFENCE	OUT_OF_GEOFENCE, 1,112 m	PASS
ATT-06	Correct OTP, GPS accuracy 250 m	LOW_ACCURACY	LOW_ACCURACY	PASS
ATT-07	Member, correct OTP, inside geofence	ACCEPTED	ACCEPTED, 11 m from centre	PASS
ATT-08	Same member checks in again	DUPLICATE	DUPLICATE	PASS
ATT-09	Non-member checks in to a club session	NOT_ELIGIBLE	NOT_ELIGIBLE	PASS
ATT-10	Second student uses the first student's device (smoke test)	DEVICE_REUSED + coach alert	DEVICE_REUSED	PASS
ATT-11	Non-numeric OTP	HTTP 400	400	PASS
ATT-12	Coach adds a member manually	Record MANUAL, no coordinates	MANUAL, latitude = null	PASS